Chapter 1 – Introduction

1.1 Introduction to Artificial Intelligence and Machine Learning

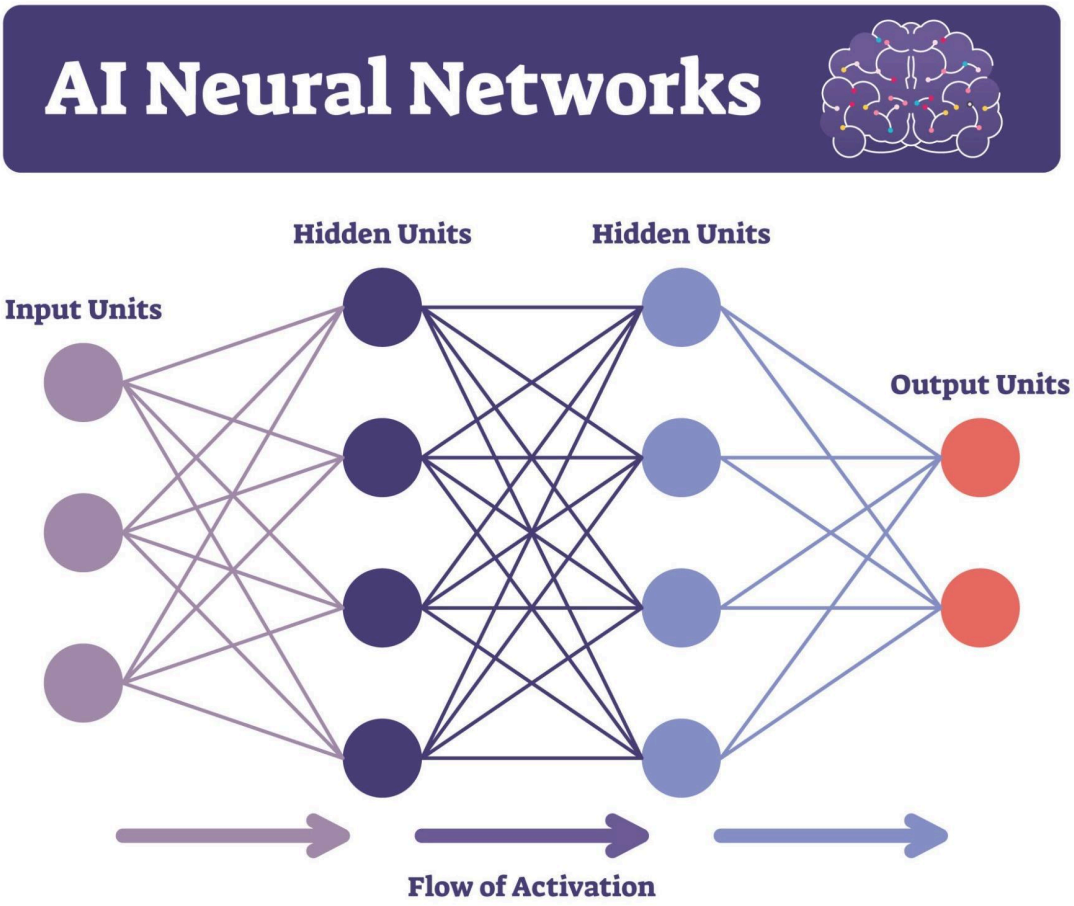
We stand at the precipice of a new era in computation, an era defined by **Artificial Intelligence (AI)**. At its core, AI is not about recreating the human mind, but about simulating human *intelligence*—a broad field of computer science dedicated to building systems that can perceive their environment, reason about it, learn from it, and take appropriate actions to achieve a specific goal. It is the engine behind tasks that were once the exclusive domain of human cognition: playing complex games like Go, translating languages in real-time, and even composing music.

Within this vast universe of AI, a star shines particularly bright: **Machine Learning (ML)**. ML represents a fundamental shift in programming. Instead of a human developer writing explicit, step-by-step rules for every possible contingency (a brittle and impossible task for complex problems), an ML model *learns* these rules for itself. It does this by ingesting and identifying statistical patterns within vast amounts of data.

We can think of this as the difference between giving someone a fish (traditional programming) and teaching them how to fish (Machine Learning).

- **AI** is the overall concept of a machine that can solve a problem.
- **ML** is the *fishing rod and reel*—the tools that allow the machine to learn from a lake of data.
- **Deep Learning (DL)**, a subfield of ML, is the advanced, self-adjusting "smart" lure that can learn to target specific, complex fish on its own.

This project is built upon Deep Learning. DL utilizes complex architectures called Artificial Neural Networks (ANNs), which are inspired by the layered, interconnected neurons in the human brain.



Getty Images

These networks are "deep," meaning they have multiple layers. Each layer learns to identify progressively more complex features from raw data. In the context of this project:

- For an **image**, the first layer might learn to see simple edges and colors. The next layers might combine these to recognize textures and shapes (a tire, a smile). The final layers might assemble these shapes to understand a complex scene (a flat tire on a car, a person celebrating).
- For **text**, the first layer might learn word representations. The next layers learn to understand the relationship between words (grammar and syntax). The final layers learn to grasp the abstract meaning of a full sentence or paragraph.

This hierarchical learning ability is precisely what allows DL models to understand the nuanced and abstract concept of *sentiment*.

1.2 Importance of Sentiment Analysis in the Modern World

The digital transformation has created an ocean of data. Every second of every day, an estimated 2.5 quintillion bytes of data are generated. This data is not in neat spreadsheets; it is the chaotic, unstructured, and deeply human text of our collective consciousness: 6,000 tweets per second, 95 million photos uploaded to Instagram daily, and 720,000 hours of video added to YouTube.

This digital deluge is a goldmine. It contains the raw, unfiltered opinions, feelings, and attitudes of billions of people. **Sentiment Analysis**, also known as opinion mining, is the computational shovel and sieve used to dig into this mountain of data and extract the invaluable gold of human *opinion*.

In the 21st century, sentiment analysis is not a niche academic pursuit; it is a critical tool for decision-making across nearly every industry:

- **Business Intelligence & Brand Health:** A company like Nike or Apple no longer has to wait for quarterly focus groups. They can, in real-time, monitor the global sentiment surrounding a new product launch. They can see *why* customers are unhappy (e.g., "the battery drains too fast") or *why* they are delighted (e.g., "the camera is amazing") and react in hours, not months.
- **Finance & Market Prediction:** Financial markets are driven by human emotion—fear and greed. Algorithmic trading bots constantly "read" financial news and social media, gauging sentiment on a stock or the market as a whole. A sudden spike in negative sentiment can trigger a sell-off fractions of a second before a human trader even finishes reading the headline.
- **Governance & Public Policy:** Governments and NGOs can monitor public pulse. During a public health crisis (like a pandemic), they can track the spread of misinformation and public anxiety. After a natural disaster, they can filter social media to find urgent requests for help.
- **Social Good & Mental Health:** Researchers are increasingly using sentiment analysis to study online expression. By analyzing linguistic markers in a user's posts (with consent), models can identify patterns associated with depression or anxiety, offering a potential new avenue for early, non-intrusive mental health screening.

In short, sentiment analysis provides a data-driven framework for understanding human emotion at a scale previously unimaginable, moving us from "What are people saying?" to "How do people *feel*?"

1.3 Evolution from Text Sentiment Analysis to

Multimodal (Image + Text) Analysis

The journey of sentiment analysis began, logically, with **text**. Early systems were simple but clever, relying on vast, hand-curated dictionaries (or "lexicons") of positive and negative words. A system might assign "+1" for "good" and "-1" for "bad." However, these systems were easily broken. They couldn't understand:

- **Negation:** "This is *not* good." (Would be read as Positive).
- **Context:** "This movie is *sick*!" (Would be read as Negative).
- **Subtlety:** "The plot was... fine." (Would be read as Neutral, missing the faint negativity).

The first great leap forward came with classical ML models (e.g., Naive Bayes, SVMs) that learned from "bags of words." The true revolution, however, was the arrival of Deep Learning models like **BERT (Bidirectional Encoder Representations from Transformers)**. BERT reads an entire sentence at once, allowing it to understand context, ambiguity, and negation. It became the gold standard for text-based sentiment analysis.

But a critical, fundamental blind spot remained. Human communication on the modern web is not just text. It is a rich, layered, and increasingly visual experience. On platforms like Instagram, X, and Reddit, the image is not just decoration; it is *part of the message*.

This creates a "context gap" that text-only models cannot cross. An image can fundamentally **invert, amplify, or disambiguate** the sentiment of the text.

- **Inversion (Sarcasm):**
  - **Text:** "My vacation is off to a great start." (Sentiment: **Positive**)
  - **Image:** A picture of an overflowing toilet in a flooded hotel room.
  - **True Multimodal Sentiment: Overwhelmingly Negative**.
- **Amplification:**
  - **Text:** "We finally did it." (Sentiment: **Neutral/Mildly Positive**)
  - **Image:** A photo of a person holding a marathon medal, crying with joy.
  - **True Multimodal Sentiment: Ecstatic/Very Positive**.
- **Disambiguation (Context):**
  - **Text:** "This is killing me!" (Sentiment: **Negative**)
  - **Image:** A screenshot of a hilarious internet meme.
  - **True Multimodal Sentiment: Positive (Amusement)**.

This project is born from this gap. The shift to **Multimodal Sentiment Analysis** is the necessary next step, an approach that processes and *fuses* information from both text and images to achieve a far more accurate and human-like understanding of meaning.

Figure 1.1: The Multimodal Context Gap

[A diagram visually contrasting text-only vs. multimodal analysis.

- **Side 1 (Text-Only):** Shows the text "My day is great." leading to a "Positive" classification.
- **Side 2 (Multimodal):** Shows the *same text* plus an *image of a flat tire*. The two inputs are fed into a "Fusion Model," which leads to a "Negative" classification.
- **Caption:** A text-only model is "deceived" by the positive word "great." The multimodal model uses the visual context from the image to correctly identify the sarcastic (negative) intent.]

1.4 Real-World Applications

When a system can understand both *what is written* and *what is shown*, it unlocks a new, more sophisticated class of applications.

- **Advanced Brand Reputation Management:** Imagine a "Brand Health Dashboard." A marketing executive at a car company can see a live feed of social media posts. The system doesn't just flag text mentions. It automatically sorts posts:
  - **Positive:** A user posts "My new ride!" with a photo of their shiny new car.
  - **Negative:** A user posts "My new ride!" with a *video* of the engine smoking.
  - This allows a customer service team to *immediately* engage the negative poster, turning a public relations disaster into a public display of good service.
- **Nuanced Social Media Monitoring:** During a large public protest, a text-only system is overwhelmed by noise. A multimodal system can categorize posts with far greater intelligence:
  - It can differentiate between a user posting "This is chaos!" with a photo of a peaceful

march (Hyperbole) and a user posting the *same text* with a photo of a burning car (Urgent, Negative).
    - NGOs and journalists can use this to get a more accurate, on-the-ground sense of the situation, filtering out rumor from reality.
- **Smarter E-commerce Reviews:** An Amazon product manager wants to know why a product has mixed reviews. A multimodal system analyzes the reviews:
    - It finds that text-only reviews are 90% positive.
    - However, it finds that reviews *with images* are 60% negative.
    - It presents the manager with a "Visual Summary" showing that customers are consistently posting photos of the product arriving with a broken part. The problem isn't the product; it's the *packaging*, an insight the text-only system completely missed.
- **Intelligent Political Opinion Tracking:** During an election, an analyst wants to gauge a candidate's support. A text-only model flags "Big crowd at the rally" as positive. Our multimodal system can differentiate:
    - **Positive:** The text is paired with a photo of a packed stadium.
    - **Negative (Sarcasm):** The same text is paired with a photo of a handful of people in an empty room.
    - This provides a powerful "truth filter" against propaganda and spin.

1.5 Problem Statement

The expressive power of human communication is increasingly multimodal. On the platforms that define the modern public square, sentiment is conveyed through a complex interplay of text, imagery, and context.

Traditional sentiment analysis systems, which are "blind" to all information outside of the text, are fundamentally ill-equipped for this reality. They fail to capture the complete message, leading to critical errors in understanding. These systems are easily "deceived" by sarcasm, fail to grasp the emotional weight added by an image, and cannot use visual cues to resolve textual ambiguity. This results in inaccurate data, flawed insights, and a distorted view of public opinion.

This project addresses the problem of **inaccurate and incomplete sentiment classification in a visually-driven digital world.** The core challenge is to design, implement, and evaluate a unified deep learning system that can effectively and automatically extract, interpret, and **intelligently fuse** sentiment-bearing features from both textual and visual modalities. The goal is to produce a single, highly accurate sentiment polarity (Positive, Negative, or Neutral)

that reflects the *true, combined meaning* of the image-text pair.

1.6 Objectives of the Project

To successfully address the problem statement, this project will be guided by the following six primary objectives:

1. **To Investigate** and conduct a comprehensive review of existing academic literature on both unimodal (text-only, image-only) and state-of-the-art multimodal sentiment analysis to identify successful techniques, established benchmarks, and existing research gaps.
2. **To Design** a robust and replicable data preprocessing pipeline capable of cleaning, normalizing, and transforming two distinct data streams—raw text and raw images—into a format suitable for ingestion by deep learning models.
3. **To Implement** and fine-tune two separate, state-of-the-art deep learning models for unimodal feature extraction: a **Transformer-based model (e.g., BERT)** to capture the rich semantic context of the text, and a **Convolutional Neural Network (e.g., ResNet)** to capture the salient visual features of the image.
4. **To Develop** and experiment with an effective feature fusion module. This module will be responsible for taking the two high-dimensional feature vectors (one from BERT, one from ResNet) and combining them into a single, comprehensive representation that captures the unified sentiment.
5. **To Train and Rigorously Evaluate** the final multimodal model on a benchmark dataset. This evaluation will involve measuring its performance using standard metrics (Accuracy, Precision, Recall, F1-Score) and critically comparing it against text-only and image-only baseline models to *quantify* the value of multimodality.
6. **To Develop** a proof-of-concept prototype, such as a simple web application. This will demonstrate the system's practical applicability by allowing a user to input their own image and text pair and receive a real-time sentiment prediction.

1.7 Scope and Limitations

To ensure the project is focused and achievable, its scope and limitations must be clearly defined.

Scope:

- **Modalities:** The project is strictly **bimodal**, focusing *only* on **static images** (e.g., .jpg, .png) and their **accompanying text captions**. Other modalities such as video, audio, memes (with embedded text), and GIFs are considered outside the current scope.
- **Language:** The Natural Language Processing (NLP) component will be developed and trained exclusively for the **English language**.
- **Sentiment Classification:** The model's output will be a **ternary** classification: **Positive, Negative, or Neutral**. While the architecture could be extended to fine-grained classification (e.g., 1-5 stars or emotions like "Angry," "Joyful"), this is not a primary objective.

Limitations:

- **Data Dependency & Bias:** This is the most significant limitation. A machine learning model is a product of its data. If the training dataset (e.g., scraped from Twitter) contains biases, the model *will* learn and replicate them. For example, if the dataset inadvertently associates certain objects, fashion styles, or cultural symbols with negative sentiment, the model will develop this same harmful bias.
- **Subjectivity of Sentiment:** Sentiment is not a hard science; it is deeply subjective. The "ground truth" labels in any dataset represent a consensus, not a universal fact. One person's "Neutral" is another's "Mildly Negative." The model's performance is measured against this imperfect consensus, and it will inevitably fail to match every individual's interpretation.
- **Computational Cost:** Training large-scale Transformer (BERT) and CNN (ResNet) models is exceptionally computationally expensive, requiring powerful GPUs and significant time. This may limit the number of experiments and the extent of hyperparameter tuning that can be performed.
- **Implicit & External Context:** The model will "see" *only* the image and text provided. It has no access to wider "world knowledge" or implicit context, such as:
    - *User History:* Is this user known for being sarcastic?
    - *Current Events:* Is there a global event (e.g., a holiday) that changes the meaning of the post?
    - *Platform Nuance:* Is this an Instagram post (where text is secondary) or a X (Twitter) post (where text is primary)?

1.8 Expected Outcomes

Upon successful completion of this project, the following deliverables will be produced:

1. **This comprehensive research document:** A detailed report (approx. 120 pages) chronicling the entire project lifecycle, from initial research and system design to final evaluation and discussion, serving as a complete academic and technical reference.
2. **A Trained Multimodal Model:** A fully trained, saved, and validated model file (.pth or .h5) that demonstrably outperforms text-only baselines and is capable of making predictions on new data.
3. **A Functional Prototype:** A proof-of-concept web application (e.g., built with Flask/FastAPI) that allows a user to interact with the trained model in real-time.
4. **All Source Code:** A well-documented code repository containing all scripts for data preprocessing, model definition, training, evaluation, and the prototype application.

1.9 Project Overview Diagram

The system is architected as a "dual-stream" or "two-branch" pipeline. The two modalities (text and image) are processed in parallel in their own specialized streams before being intelligently fused at the end for a final, unified decision. The high-level workflow is visualized in Figure 1.2.

Figure 1.2: High-Level System Architecture

[A detailed, clean, and professional-looking workflow diagram.

1. **Inputs:** An **Image Input** (e.g., a photo of a flat tire) and a **Text Input** (e.g., "My day is going just great.") are shown at the top.
2. **Stream 1 (Text):** The Text Input box points to a block labeled "**Text Preprocessing Module**" (functions: Tokenization, Cleaning, Padding). This block points to a larger block labeled "**Text Feature Extractor (BERT)**," which outputs a box labeled "**Text Feature Vector (768-dim)**".
3. **Stream 2 (Image):** In parallel, the Image Input box points to a block labeled "**Image Preprocessing Module**" (functions: Resize to 224x224, Normalization). This block points to a larger block labeled "**Image Feature Extractor (ResNet-50)**," which outputs a box labeled "**Image Feature Vector (2048-dim)**".
4. **Fusion:** Both the Text Feature Vector and the Image Feature Vector point to a central block labeled "**Feature Fusion Module**" (function: Concatenation). This outputs a single, larger box labeled "**Fused Vector (2816-dim)**".
5. **Classification:** The Fused Vector box points to a final block labeled "**Sentiment Classifier (MLP Head)**".
6. **Output:** This classifier block points to the final output, a box labeled "**Sentiment Prediction** (e.g., 'Negative')".

- **Caption:** The proposed system architecture. Text and image data are processed in parallel by specialized deep learning models (BERT, ResNet) to extract high-dimensional feature vectors. These vectors are then concatenated and passed to a classifier head, which makes the final sentiment prediction.]

Chapter 2: Literature Review

Goal: Explore What's Already Been Done and Identify

Your Unique Contribution

This chapter reviews the existing body of knowledge on sentiment analysis, tracing its evolution from text-based origins to the complex, emerging field of multimodal sentiment analysis. The goal is to provide a comprehensive overview of the historical context, major research approaches, and dominant techniques that have shaped the discipline. By critically examining the capabilities and limitations of unimodal (text-based and image-based) analysis, we establish a clear rationale for the shift toward multimodal approaches. This review will culminate in the identification of a significant research gap in the field—specifically, the challenges in data fusion and contextual understanding—that this dissertation aims to address.

2.1. Historical Development of Sentiment Analysis

Sentiment analysis, also known as opinion mining, is a field of computational study focused on identifying and extracting subjective information from source materials. Its roots are intertwined with the development of computational linguistics and natural language processing (NLP).

The earliest forms of sentiment analysis can be traced back to the mid-20th century, with initial forays into computational methods for understanding language (e.g., in the 1950s) focusing on rule-based systems. However, the field truly began to formalize in the 1990s and early 2000s, driven by the explosive growth of the internet and Web 2.0. The rise of blogs, forums, and e-commerce sites like Amazon and eBay created a massive, publicly accessible corpus of user-generated opinions in the form of product reviews.

Early research, such as the seminal work by Turney (2002) on "Thumbs Up or Thumbs Down?" and Pang, Lee, and Vaithyanathan (2002) on sentiment classification of movie reviews, established the foundational task. These initial approaches primarily used machine learning techniques and lexicon-based methods.

- **Lexicon-Based Approaches:** These were the earliest methods, relying on predefined dictionaries (lexicons) of words tagged with positive or negative "sentiment polarity." A document's overall sentiment was calculated by aggregating the scores of its constituent

words. While simple and interpretable, these methods struggled with context, negation ("not good"), and sarcasm.

- **Machine Learning Approaches:** This paradigm shift involved training classifiers on labeled datasets. Researchers began using algorithms like Naive Bayes, Support Vector Machines (SVM), and Maximum Entropy to classify a document as positive, negative, or neutral. This approach was more robust as the model could "learn" patterns beyond simple word-matching.

The historical development was thus characterized by a move from manually crafted rules and dictionaries to statistical learning models that could better handle the ambiguity of human language.

2.2. Review of Major Research Papers and Approaches

The evolution of sentiment analysis is best understood through its three main methodological eras: lexicon-based, traditional machine learning, and deep learning.

Lexicon and Rule-Based Approaches:
As mentioned, early work heavily relied on sentiment lexicons. Notable resources include SentiWordNet, which assigned positivity, negativity, and objectivity scores to WordNet synsets, and the General Inquirer (GI) lexicon. These approaches were effective for straightforward domains but lacked the flexibility to handle nuanced or domain-specific language.
Traditional Machine Learning Approaches:
The work of Pang, Lee, and Vaithyanathan (2002) is a cornerstone paper. They applied Naive Bayes (NB), Maximum Entropy (ME), and Support Vector Machines (SVM) to the task of movie review classification, finding that SVMs generally outperformed other methods. Their work demonstrated that sentiment analysis could be effectively treated as a text classification problem. For years, the standard pipeline involved:

1. **Data Preprocessing:** Tokenization, stop-word removal, and stemming/lemmatization.
2. **Feature Extraction:** Creating numerical representations of text, most commonly using **Bag-of-Words (BoW)** or **TF-IDF (Term Frequency-Inverse Document Frequency)**, which represent documents as vectors based on word counts.

3. **Model Training:** Feeding these vectors into an ML classifier like SVM.

This paradigm dominated research for nearly a decade, with subsequent work focusing on improved feature engineering (e.g., using n-grams) and adapting models to different domains (e.g., political science, finance).

Deep Learning Approaches:
The 2010s marked a revolutionary shift with the resurgence of deep learning. Neural networks demonstrated an ability to automatically learn hierarchical features from raw data, eliminating the need for manual feature engineering.

- **Word Embeddings (Word2Vec, GloVe):** Papers by Mikolov et al. (Word2Vec) and Pennington et al. (GloVe) introduced a way to represent words as dense vectors in a continuous space. These embeddings captured semantic relationships (e.g., "king" - "man" + "woman" $\approx$ "queen"), providing models with a much richer understanding of language than sparse BoW vectors.
- **Recurrent Neural Networks (RNNs) and LSTMs:** Kim (2014) showed that even a simple Convolutional Neural Network (CNN) trained on top of pre-trained word embeddings could achieve state-of-the-art results on several sentiment classification tasks. Concurrently, Recurrent Neural Networks (RNNs), and more specifically their gated variants like **Long Short-Term Memory (LSTM)** and **Gated Recurrent Units (GRU)**, became popular. Their sequential nature made them ideal for modeling text, as they could maintain a "memory" of previous words in a sentence, theoretically capturing context and long-range dependencies.

These deep learning models became the new state-of-the-art, as they were far more effective at understanding the complex, contextual, and sequential nature of language.

2.3. Comparative Study: Text-based vs. Image-based Sentiment Analysis

With the rise of social media platforms like Instagram, Facebook, and Twitter, communication became increasingly visual. This led to the emergence of a parallel field: **visual sentiment analysis**, which attempts to infer sentiment directly from images.

Text-Based Sentiment Analysis:

- **Strengths:**
  - **Explicitness:** Sentiment is often stated directly and unambiguously (e.g., "I love this product," "This movie was terrible").
  - **Maturity:** As a field, it is highly mature with robust, well-understood models and vast

labeled datasets.
    - **Granularity:** Capable of **Aspect-Based Sentiment Analysis (ABSA)**, which can identify sentiment toward specific *aspects* of an entity (e.g., "The *camera* is great, but the *battery life* is awful").
- **Weaknesses:**
    - **Ambiguity:** Struggles with sarcasm, irony, and figurative language ("That's sick!" could be positive or negative).
    - **Context-Dependence:** The meaning of a text can be entirely dependent on external context not present in the words themselves.
    - **Implicit Sentiment:** Fails when sentiment is implied rather than stated.

Image-Based (Visual) Sentiment Analysis:

- **Strengths:**
    - **Universality:** Visual cues (e.g., a smile, a beautiful sunset, a car crash) can convey emotions that often transcend language barriers.
    - **Implicit Cues:** Can capture subtle, non-verbal expressions of sentiment, such as body language, facial expressions, or the aesthetic quality and emotional tone of a scene (e.g., via color, lighting, and composition).
- **Weaknesses:**
    - **Subjectivity:** Highly subjective. A picture of a spider might evoke fear in one person and fascination in another. A picture of a steak is positive for a meat-eater but negative for a vegetarian.
    - **Context-Dependence:** The most significant weakness. An image of a person crying could represent sadness (at a funeral) or joy (at a wedding). Without context, the true sentiment is unknowable.
    - **Nascent Field:** Less mature than text-based analysis, with more complex feature extraction and a greater scarcity of high-quality, large-scale labeled datasets (e.g., "this image is positive").

Comparative Conclusion:
Both unimodal approaches are powerful but fundamentally incomplete. Text-based analysis misses the rich, implicit emotional cues present in visuals, while image-based analysis is plagued by ambiguity and a desperate need for context. On social media, users frequently combine these modalities, using an image to show a scene and text to provide the explicit context or sentiment (e.g., a photo of a crowded beach with the text, "I miss this!" vs. "My nightmare."). This inherent complementarity is the primary driver for multimodal sentiment analysis.

2.4. Deep Learning and NLP Techniques Overview

The current landscape of NLP is dominated by the **Transformer** architecture, introduced by

Vaswani et al. (2017) in the paper "Attention Is All You Need."

- **The Transformer and Self-Attention:** Unlike RNNs, which process text sequentially, the Transformer uses a mechanism called **self-attention** to process all words in a sentence simultaneously. It "weighs" the importance of all other words in the sentence relative to a given word, allowing it to build a highly contextual representation. This parallelizable and powerful architecture quickly made RNNs/LSTMs obsolete for most state-of-the-art tasks.
- **Pre-trained Language Models (PLMs):** The true breakthrough came with the application of transfer learning to the Transformer architecture.
    - **BERT (Bidirectional Encoder Representations from Transformers):** Devlin et al. (2018) released BERT, a model pre-trained on a massive unlabeled text corpus (like all of Wikipedia) using a "Masked Language Model" objective. BERT learns a deep, bidirectional understanding of language. This pre-trained model can then be **fine-tuned** on a much smaller, task-specific labeled dataset (like sentiment analysis) to achieve state-of-the-art results with minimal training.
    - **RoBERTa, ALBERT, and GPT:** BERT was quickly followed by numerous variants like RoBERTa (a "robustly optimized" BERT) and ALBERT (a "lite" BERT), as well as decoder-based models like the Generative Pre-trained Transformer (GPT) series, which excels at text generation.

For modern sentiment analysis, the standard approach is no longer to train a model from scratch but to fine-tune a pre-trained Transformer model like BERT or RoBERTa.

2.5. Transfer Learning in Vision Models

A parallel revolution occurred in computer vision, and it actually pre-dated the one in NLP. The key event was the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)**.

- **Convolutional Neural Networks (CNNs):** Models like **AlexNet** (2012), **VGGNet**, **GoogLeNet**, and later **ResNet (Residual Networks)** demonstrated that deep CNNs, trained on the massive ImageNet dataset (with 1.2 million labeled images), could dramatically outperform all previous methods for image classification.
- **Transfer Learning in Vision:** Researchers quickly discovered that these models weren't just learning to classify 1000 object types. The early layers of the network learned to

detect universal, low-level features like edges, corners, and textures. The deeper layers learned to combine these into more complex patterns like faces, wheels, and fur.

This allowed for highly effective **transfer learning**:

1. Take a state-of-the-art CNN (e.g., ResNet-50) pre-trained on ImageNet.
2. Remove its final classification layer (which was specific to the 1000 ImageNet classes).
3. Use the "headless" model as a fixed **feature extractor**. When fed a new image, it outputs a dense vector (a feature embedding) that represents the image's content.
4. This feature vector can then be fed into a simple classifier (like SVM or a small neural network) to train it on a new task, like visual sentiment, even with a small dataset.

This pre-training/fine-tuning paradigm is the undisputed standard for virtually all modern computer vision tasks, including the visual component of multimodal sentiment analysis.

2.6. Multimodal Sentiment Analysis: Research Gap

Multimodal Sentiment Analysis (MSA) aims to create a holistic understanding of sentiment by integrating information from multiple modalities, most commonly text, vision, and acoustics (speech). The central hypothesis is that these modalities provide complementary information, and their combination will lead to a more accurate and robust model.

Major Approaches and Models:
Early work focused on simple fusion techniques, such as:

- **Early Fusion (Feature-Level):** Concatenating the feature vectors from each modality (e.g., a BERT vector for text and a ResNet vector for the image) and feeding this combined vector into a single classifier.
- **Late Fusion (Decision-Level):** Training separate classifiers for each modality and then combining their output predictions (e.g., through averaging or a weighted vote).

More recent research, as highlighted in systematic reviews, focuses on more complex fusion mechanisms that allow the modalities to interact and influence each other's representations. This includes:

- **Attention Mechanisms:** Cross-modal attention, where, for example, the text model can "attend to" relevant parts of the image, and vice-versa.
- **Transformer-Based Fusion:** Large-scale, pre-trained multimodal models like **ViLBERT** and **LXMERT** (which are pre-trained on massive text-image datasets) have shown strong performance, but are often trained on tasks like visual question answering, not sentiment.
- **Tensor-Based Fusion:** Using tensor operations to model complex, high-order interactions between modalities.

The Research Gap:

Despite this progress, significant challenges—and thus, research gaps—remain.

1. **Semantic Alignment and Fusion:** This is the most critical gap. How do you effectively "fuse" a word like "beautiful" with the visual features of a sunset? Modalities are not always synchronous or complementary. They can be **conflicting** (e.g., a smiling photo with the text "I'm so miserable"). This is the problem of **sarcasm and incongruity**, which most models handle poorly. Current fusion methods often struggle to model these complex cross-modal interactions and determine which modality to "trust."
2. **Contextual Understanding:** Most models are trained on curated datasets (e.g., Twitter, MVSA) but fail to generalize to "in-the-wild" data. The sentiment of a post can be entirely dependent on real-world events, cultural context, or a personal narrative that is not explicitly present in either the text or the image.
3. **Data Annotation and Bias:** Creating large-scale, high-quality labeled multimodal datasets is extremely expensive and difficult. This leads to models being trained on smaller, often noisy datasets, which can inherit and amplify biases present in the data (e.g., associating certain visual or linguistic patterns with negative sentiment incorrectly).
4. **Real-Time Implementation:** Advanced fusion models like cross-modal Transformers are computationally expensive, making them difficult to deploy in real-time applications (e.g., content moderation).

This dissertation will focus on the primary gap: **semantic alignment and fusion, particularly in the presence of incongruity between modalities.** While attention and transformer models provide a powerful framework, they are not inherently designed to model dissonance. The central research question is: can we develop a fusion architecture that not only combines complementary information but also explicitly identifies and reasons about *conflicting* information between text and images to better detect complex sentiments like sarcasm and irony?

2.7. Summary of Findings from Existing Literature

This literature review has traced the evolution of sentiment analysis from its origins in text-based, lexicon-driven systems to the current era of deep learning and multimodality.

- **Historical Progression:** The field has progressed from lexicon-based rules to traditional machine learning (SVMs with BoW/TF-IDF) and finally to deep learning models (LSTMs, CNNs).
- **The Transformer Revolution:** In both NLP and vision, the dominant paradigm is now **transfer learning**. Pre-trained Transformer models (like BERT for text) and CNNs (like ResNet for vision) are fine-tuned for specific tasks, serving as powerful feature extractors.
- **Unimodal Limitations:** We have established that both text-only and image-only sentiment analysis are fundamentally limited. Text struggles with implicit sentiment and ambiguity, while images are hamstrung by subjectivity and a critical lack of context.
- **The Multimodal Imperative:** The frequent co-occurrence of text and images on social media, where one modality often provides context for the other, creates a clear imperative for multimodal models.
- **The Research Gap:** The primary challenge and research gap in Multimodal Sentiment Analysis is not simply *how* to combine modalities, but how to model their complex, non-linear interactions. This includes handling cases of **incongruity and conflict** (like sarcasm) where the sentiment is not a simple sum of its parts. Current models excel at complementary fusion but often fail when modalities are dissonant.

This review confirms that a significant opportunity exists to develop a more sophisticated fusion mechanism that can better model the full spectrum of multimodal interactions, from complementary to conflicting, to achieve a more robust and human-like understanding of sentiment.

Chapter 3: Theoretical Background

This chapter provides a comprehensive review of the fundamental concepts and advanced models that form the theoretical bedrock of this research. We delve into the core principles of Natural Language Processing (NLP) and Computer Vision (CV), exploring the techniques used to process and understand textual and visual data, respectively. Subsequently, we examine the sophisticated field of multimodal learning, which focuses on integrating these two disparate data streams. Finally, we review the specific deep learning architectures and sentiment classification techniques that are pivotal to the methodologies employed in this study.

3.1 Natural Language Processing (NLP)

Natural Language Processing is a subfield of linguistics, computer science, and artificial intelligence concerned with the interactions between computers and human language. The ultimate objective of NLP is to enable computers to process, understand, and generate human language in a way that is both meaningful and useful. This requires overcoming the inherent ambiguity, context-dependency, and variability of language. The following sections detail the foundational preprocessing steps and representation techniques essential for preparing text data for machine learning models.

3.1.1 Tokenization, Stemming, and Lemmatization

Before any analysis can occur, raw text must be segmented into meaningful units. This process is known as **tokenization**. It typically involves breaking down a text corpus into individual words, phrases, symbols, or other meaningful elements called tokens. For example, the sentence "The quick brown fox jumps over the lazy dog." would be tokenized into ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog", "."]. This step is crucial as it converts a string of characters into a list of items that a model can process individually.

Once tokenized, words often need to be normalized to their base or root form. This reduces the feature space and helps the model recognize that words like "jump," "jumps," and "jumping" refer to the same underlying concept. Two common methods for this are stemming

and lemmatization.

Stemming is a rudimentary, rule-based process that chops off the ends of words (suffixes) to obtain a "stem." For instance, "jumps," "jumping," and "jumped" might all be reduced to the stem "jump." While fast and computationally inexpensive, stemming can be imprecise. It may conflate words that are not semantically related (e.g., "universe" and "university" might both be stemmed to "univers") or produce stems that are not actual words (e.g., "studies" might become "studi").

Lemmatization is a more sophisticated and linguistically-informed approach. It uses a vocabulary and morphological analysis of words to return the base or dictionary form, known as the "lemma." For example, the lemma for "is," "was," and "are" is "be." The lemma for "jumping" is "jump." Lemmatization is more accurate than stemming as it considers the context of the word and its part of speech. However, it is computationally more expensive as it requires a dictionary lookup.

3.1.2 Stop Words Removal

Languages contain many high-frequency words that carry little semantic weight, such as "a," "an," "the," "is," "in," and "on." These are known as **stop words**. In many NLP tasks like information retrieval and text classification, these words can introduce noise and add computational overhead without contributing significant meaning. Stop words removal is the process of identifying and filtering out these words from the tokenized text. While a common step, its application is task-dependent. For some tasks, like sentiment analysis, stop words can be important (e.g., "not" in "not good" is critical). For other tasks, like language modeling or machine translation, preserving the grammatical structure, including stop words, is essential.

3.1.3 TF-IDF

After preprocessing, text must be converted into a numerical format. A classic and effective method for this is **Term Frequency-Inverse Document Frequency (TF-IDF)**. TF-IDF is a numerical statistic that reflects how important a word is to a document in a collection or corpus. It is a "bag-of-words" model, meaning it represents text as an unordered collection of words, disregarding grammar and word order.

The TF-IDF value is the product of two statistics:

1. Term Frequency (TF): This measures the frequency of a word in a specific document. It is often normalized to prevent a bias towards longer documents.
   TF(t, d) = (Number of times term t appears in document d) / (Total number of terms in document d)
2. Inverse Document Frequency (IDF): This measures how much information the word provides. It is a logarithmic scaling of the inverse of the fraction of documents in the corpus that contain the word. This metric down-weights common words (like stop words)

and up-weights rare words.
IDF(t, D) = log(Total number of documents in corpus D / (Number of documents containing term t + 1))

The final TF-IDF score is calculated as:
TF-IDF(t, d, D) = TF(t, d) * IDF(t, D)
Words with a high TF-IDF score are frequent in a specific document but rare across the entire corpus, making them highly descriptive of that document's content.

3.1.4 Word Embeddings

While TF-IDF is effective, it has limitations. It results in very high-dimensional, sparse vectors (one dimension for every word in the vocabulary) and, most importantly, it fails to capture semantic relationships between words. For example, in a TF-IDF matrix, the vectors for "cat" and "dog" are as different as the vectors for "cat" and "airplane."

Word Embeddings solve this problem by representing words as dense, low-dimensional vectors (e.g., 100-300 dimensions). These vectors are learned from a large text corpus, and their positions in the vector space are arranged such that words with similar meanings are located close to one another. This allows models to understand concepts like "cat" is to "kitten" as "dog" is to "puppy," or that "king" - "man" + "woman" is close to "queen."

3.1.5 Word2Vec

Word2Vec, developed by Mikolov et al. at Google, is one of the most popular techniques for learning word embeddings. It is not a single algorithm but a family of models that includes two main architectures:

1. **Continuous Bag-of-Words (CBOW):** The CBOW model learns embeddings by predicting the current word based on its surrounding context (neighboring words). It essentially "fills in the blank."
2. **Skip-gram:** The Skip-gram model does the opposite. Given a target word, it predicts the surrounding context words. It is generally considered to be better for capturing the meaning of rare words and performs well on semantic analogy tasks.

Both models use a shallow neural network, and the learned weights of the hidden layer serve as the word embeddings.

3.1.6 GloVe

GloVe, or Global Vectors for Word Representation, is another popular embedding technique developed at Stanford. While Word2Vec is a "predictive" model that learns from local context windows, GloVe is a "count-based" model. It learns by constructing a large co-occurrence matrix that captures how frequently words appear together in the corpus. It then uses matrix factorization to reduce the dimensionality of this matrix, producing the final word embeddings. GloVe aims to combine the strengths of both count-based methods (like TF-IDF)

and predictive methods (like Word2Vec), capturing both global corpus statistics and local context.

3.1.7 BERT

A significant limitation of Word2Vec and GloVe is that they produce static, or context-independent, embeddings. The word "bank" has the same vector representation regardless of whether it appears in "river bank" or "savings bank."

BERT (Bidirectional Encoder Representations from Transformers), introduced by Google in 2018, revolutionized NLP by providing contextual embeddings. BERT uses the Transformer architecture, which employs a mechanism called "self-attention" to weigh the importance of different words in a sentence when processing a single word. BERT is "bidirectional" because it reads the entire sequence of words at once, allowing it to understand the context of a word based on all the words that surround it (both left and right).

BERT is pre-trained on a massive corpus (like all of Wikipedia) using two tasks: Masked Language Model (MLM), where it predicts randomly masked words in a sentence, and Next Sentence Prediction (NSP). The pre-trained BERT model can then be "fine-tuned" for specific downstream tasks like sentiment analysis or question answering, achieving state-of-the-art results. The output of BERT is a different vector for a word depending on its context, thus solving the "bank" problem.

3.2 Computer Vision (CV)

Computer Vision is a field of AI that trains computers to interpret and understand the visual world. Using digital images from cameras and videos, models can accurately identify, classify, and react to objects. This section focuses on the dominant deep learning approach for CV: Convolutional Neural Networks.

3.2.1 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs or ConvNets) are a class of deep neural networks specifically designed to process pixel data. They are inspired by the human visual cortex and have become the standard for image-based tasks. A CNN architecture is composed of several key layers:

1. **Convolutional Layer:** This is the core building block. It uses a set of learnable filters (or kernels) that slide over the input image to perform a convolution operation. Each filter is

designed to detect a specific feature, such as an edge, a corner, or a texture. The output of this layer is a "feature map" that indicates the locations in the image where the filter's feature was detected.
2. **Activation Layer (ReLU):** After the convolution, an activation function is applied to introduce non-linearity into the model. The most common activation is the Rectified Linear Unit (ReLU), which replaces all negative pixel values in the feature map with zero. This helps the network learn complex patterns and speeds up training.
3. **Pooling Layer:** This layer is used to downsample the feature maps, reducing their spatial dimensions (width and height) while retaining the most important information. The most common type is "Max Pooling," which takes a small window (e.g., 2x2) and outputs only the maximum value from that window. This makes the model more robust to variations in the position of features and reduces computational load.
4. **Fully Connected Layer:** After several convolutional and pooling layers, the high-level feature maps are "flattened" into a one-dimensional vector. This vector is then fed into a standard feed-forward neural network (fully connected layers) which acts as a classifier. It uses the learned features to make a final prediction (e.g., "cat," "dog," "car").

3.2.2 Feature Extraction

The power of CNNs lies in their ability to automatically learn a hierarchy of features. This process is known as **feature extraction**. The initial convolutional layers learn simple, low-level features like edges, corners, and colors. As the data passes deeper into the network, subsequent layers combine these simple features to learn more complex, high-level features, such as shapes, textures, and object parts (e.g., "eye," "wheel," "door"). In the final layers, the network combines these parts to recognize entire objects. This automatic learning of features from raw pixels eliminates the need for manual, hand-crafted feature engineering, which was a major bottleneck in traditional computer vision.

3.2.3 Transfer Learning

Training a deep CNN from scratch requires an enormous amount of labeled data (often millions of images) and immense computational power. **Transfer Learning** is a technique that leverages a model pre-trained on a large, general-purpose dataset (like ImageNet, which has 1.2 million images across 1,000 classes) and adapts it for a new, specific task.

The logic is that the hierarchical features learned by the pre-trained model (e.g., edges, shapes, textures) are useful for many different vision tasks. The process typically involves:

1. Taking a pre-trained model.
2. Freezing the weights of the early convolutional layers (the "feature extractor").
3. Removing the original fully connected layers (the "classifier").
4. Adding new fully connected layers that are "trainable" and sized for the new task (e.g., binary sentiment classification).
5. Training (or "fine-tuning") only these new layers on the smaller, task-specific dataset.

This approach allows for the development of highly accurate models with much less data and

computation.

3.2.4 VGG16 and ResNet

VGG16 is a classic CNN architecture known for its simplicity and depth. It consists of 16 layers (13 convolutional, 3 fully connected) that exclusively use small 3x3 convolutional filters. Its uniform architecture demonstrated that a significant depth (many layers) was a crucial component for achieving good performance.

ResNet (Residual Network) introduced a novel "residual block" with a "skip connection." This connection allows the input of a layer to be added to its output, creating a shortcut. This innovation solved the "vanishing gradient" problem, which prevented earlier models from being trained effectively when they became too deep. By allowing gradients to flow more easily through the network, ResNet enabled the creation of models with hundreds or even thousands of layers (e.g., ResNet-50, ResNet-101), leading to new levels of accuracy.

3.3 Multimodal Learning

Humans experience the world multimodally—we see, hear, read, and speak, and our brains naturally integrate these different streams of information. **Multimodal Learning** is a branch of machine learning that aims to replicate this, building models that can process and relate information from multiple modalities, such as text, images, and audio. A key challenge is integrating heterogeneous data, as text and image vectors (embeddings) originate in different, unaligned spaces.

3.3.1 Combining Image and Text Embeddings

To perform tasks like visual sentiment analysis or image captioning, a model must understand the joint representation of an image and its associated text. This requires combining their respective embeddings. An image embedding, typically extracted from the penultimate layer of a CNN (like ResNet), captures visual features. A text embedding, extracted from an NLP model (like BERT), captures semantic meaning. The goal is to project these two vectors into a common, shared embedding space where semantically related images and text are located close together.

3.3.2 Fusion Techniques

Fusion is the process of combining the information from different modalities. The choice of when and how to fuse this information is a critical design decision that significantly impacts performance.

3.3.3 Early Fusion

Early Fusion, also known as feature-level fusion, involves combining the raw or low-level features from each modality at the very beginning of the modeling process. For example, the image and text vectors could be simply concatenated into a single, larger vector. This combined vector is then fed into a single, unified deep learning model for processing.

- **Pros:** Allows the model to learn complex interactions between the modalities at the in-depth feature level.
- **Cons:** The representations must be synchronized and are often of different dimensions and types, making simple concatenation challenging. It can also lead to a very high-dimensional feature space.

3.3.4 Late Fusion

Late Fusion, also known as decision-level fusion, takes the opposite approach. It involves training separate models for each modality independently. For instance, a CNN predicts a sentiment score from the image, and an LSTM predicts a sentiment score from the text. The final prediction is then made by combining the outputs (the decisions or scores) of these individual models, perhaps through a simple average, a weighted sum, or a small neural network.

- **Pros:** Simple to implement and allows for the use of pre-trained, modality-specific models. It is robust if one modality is missing or noisy.
- **Cons:** It fails to learn the intricate, low-level correlations between modalities, as the fusion happens too late in the process.

3.3.5 Hybrid/Intermediate Fusion

Hybrid Fusion (or Intermediate Fusion) offers a compromise. It involves fusing the modalities at multiple points or in the middle layers of the deep learning architecture. For example, image and text features might be processed by a few independent layers first, then combined (fused) and fed into deeper, shared layers for joint processing. This approach allows the model to learn modality-specific features in the early layers while still capturing inter-modality dependencies in the later layers. This is often the most effective but also the most complex strategy to design.

3.4 Deep Learning Models

This section covers the specific neural network architectures used for processing sequential data (text) and for integrating both visual and textual information.

3.4.1 LSTM (Long Short-Term Memory)

A **Recurrent Neural Network (RNN)** is a type of network designed for sequential data, maintaining an internal "memory" or state that captures information from previous time steps. However, simple RNNs suffer from the short-term memory problem—they struggle to retain information from many time steps back (vanishing gradient problem).

LSTMs are a specialized type of RNN that solve this problem through a more complex "cell" structure. An LSTM cell contains three "gates" (input, forget, and output gates) that act as regulators of information flow.

- The **Forget Gate** decides what information from the previous cell state to discard.
- The **Input Gate** decides what new information to store in the cell state.
- The Output Gate decides what information from the current cell state to use as the output (hidden state) for the current time step.
  This gating mechanism allows LSTMs to selectively remember or forget information over long sequences, making them highly effective for tasks like text classification, machine translation, and time-series analysis.

3.4.2 GRU (Gated Recurrent Unit)

The **GRU** is a newer, simplified variant of the LSTM. It combines the forget and input gates into a single "update gate" and merges the cell state and hidden state. This results in a model with fewer parameters than an LSTM, making it computationally faster to train. Despite its simplicity, GRUs often achieve performance comparable to LSTMs on many tasks.

3.4.3 BERT (Bidirectional Encoder Representations from Transformers)

As discussed in section 3.1.7, BERT is a landmark model in NLP. Its architecture is based entirely on the **Transformer**, which eschews recurrence (like LSTMs) in favor of **self-attention**. The self-attention mechanism allows the model to look at all other words in the sentence when encoding a specific word, generating a deeply contextualized representation. Its bidirectional nature and pre-training on a massive dataset allow it to be fine-tuned to become a powerful classifier for tasks like sentiment analysis, where it can capture subtle nuances of language.

3.4.4 Vision Transformer (ViT)

Traditionally, CNNs have been the *de facto* standard for computer vision. The **Vision Transformer (ViT)**, however, demonstrated that the Transformer architecture could be

successfully applied to images. To do this, a ViT first splits an input image into a sequence of fixed-size, non-overlapping patches (e.g., 16x16 pixels). Each patch is then linearly "embedded" (flattened) into a vector. These patch embeddings are treated as a sequence of tokens, much like words in a sentence. A standard Transformer encoder, complete with self-attention, is then used to process this sequence and classify the image. ViT models, when pre-trained on very large datasets, have been shown to match or exceed the performance of state-of-the-art CNNs.

3.4.5 CLIP (Contrastive Language-Image Pre-Training)

CLIP, developed by OpenAI, is a powerful multimodal model that learns the connection between images and text. It is pre-trained on a massive, noisy dataset of 400 million image-text pairs scraped from the internet. CLIP consists of two separate encoders: one for images (like a ViT or ResNet) and one for text (a Transformer).

During training, the model is given a batch of image-text pairs and learns to predict which text caption corresponds to which image. It does this using a "contrastive" loss function: it tries to maximize the similarity (e.g., cosine similarity) between the embeddings of the *correct* image-text pairs while minimizing the similarity between *incorrect* pairs.

The result is two encoders that project images and text into a shared, aligned embedding space. This allows CLIP to perform "zero-shot" classification. For example, to classify an image as "dog" or "cat," one can simply embed the image and also embed the text prompts "a photo of a dog" and "a photo of a cat." The model then predicts the class corresponding to the text prompt that has the highest cosine similarity to the image embedding, all without ever being explicitly trained on that specific task.

3.5 Sentiment Classification Techniques

Sentiment classification (or sentiment analysis) is the task of using NLP and machine learning to identify, extract, and quantify subjective information in text or other data. It aims to determine the attitude or emotional tone of the speaker, writer, or subject.

3.5.1 Binary Classification

This is the most common and straightforward form of sentiment classification. The goal is to categorize a piece of text into one of two mutually exclusive classes. In the context of sentiment, this is typically **Positive** or **Negative**. For example, a movie review is classified as either a "thumbs up" or "thumbs down." This is a standard supervised learning problem where a model is trained on a dataset of texts pre-labeled as positive or negative.

3.5.2 Ternary Classification

Binary classification often fails to capture the nuance of neutral statements. **Ternary Classification** extends the binary model by adding a third class: **Neutral**. The classes become **Positive**, **Negative**, and **Neutral**. This is particularly useful for analyzing news articles, product reviews, or social media posts where many statements are purely factual or objective and carry no strong sentiment. Identifying neutral text can be just as important as identifying polar sentiment, as it helps filter out noise and focus on strongly opinionated content.

3.5.3 Fine-Grained Classification

In many real-world applications, a simple positive/negative/neutral scale is insufficient. For instance, product review systems often use a 5-star rating scale. **Fine-Grained Classification** addresses this by breaking down the sentiment into a more detailed spectrum. Common examples include:

- **Very Positive** (5 stars)
- **Positive** (4 stars)
- **Neutral** (3 stars)
- **Negative** (2 stars)
- **Very Negative** (1 star)

This approach provides a much more detailed understanding of user sentiment but is also a more challenging machine learning problem, as the distinctions between adjacent classes (e.g., 4-star vs. 5-star) can be very subtle.

Chapter 4: System Analysis and Design

Goal: Explain how the project is planned, structured, and designed.

Following the theoretical exploration of Natural Language Processing and Computer Vision in Chapter 3, this chapter transitions from theory to practice. It lays the complete architectural and design blueprint for the proposed Multimodal Sentiment Analysis (MSA) system. The purpose of this chapter is to meticulously define the system's requirements, structure, data flows, and constituent modules.

This systematic design process is critical for ensuring that the final implementation is robust, scalable, and, most importantly, correctly addresses the research gap identified in Chapter 2. We will move from high-level requirement analysis to a low-level, module-by-module explanation, providing a clear and reproducible plan for the system's construction.

4.1 Requirement Analysis

Requirement analysis is the foundational stage of system design. It involves defining the expectations and constraints of the system from the perspective of its users and stakeholders. This is divided into two categories: functional requirements (what the system *does*) and non-functional requirements (how the system *is*).

Functional Requirements

Functional requirements define the specific behaviors, services, and functions the system must provide.

- **F.R.1: Data Ingestion:** The system must be able to accept input from a user in two forms:
  - A text-based component (e.g., a tweet, a caption, a review).
  - An image component (e.g., a corresponding photograph, a meme, a screenshot).
- **F.R.2: Unimodal Input Handling:** The system must be able to gracefully handle inputs where one modality is missing (e.g., text-only or image-only) and perform unimodal sentiment analysis in such cases.
- **F.R.3: Text Pre-processing:** The system shall automatically pre-process all text input. This includes, but is not limited to, tokenization, conversion to lowercase, and preparation for the feature extraction model (e.g., WordPiece tokenization for BERT).
- **F.R.4: Image Pre-processing:** The system shall automatically pre-process all image input. This includes resizing the image to the required input dimensions of the vision model (e.g., 224x224 pixels), normalization of pixel values, and tensor conversion.
- **F.R.5: Unimodal Feature Extraction:** The system must extract high-level feature vectors from each modality independently.
  - **Text:** Generate a contextual embedding vector (e.g., a 768-dimension vector from a pre-trained BERT model).
  - **Image:** Generate a dense feature vector (e.g., a 2048-dimension vector from a pre-trained ResNet-50 model).
- **F.R.6: Multimodal Fusion:** The system must implement a fusion mechanism to combine the text feature vector and the image feature vector into a single, unified, multimodal representation.
- **F.R.7: Sentiment Classification:** The system must feed the fused vector into a classifier (e.g., a feed-forward neural network) to produce a final sentiment prediction.
- **F.R.8: Output Display:** The system must present the final sentiment classification to the user in a clear, human-readable format (e.g., "Positive," "Negative," "Neutral," "Sarcasm/Incongruent").
- **F.R.9: Analysis History (Optional):** The system should provide a mechanism for a user to view a log of their past analysis requests and the corresponding results.

Non-Functional Requirements

Non-functional requirements define the quality attributes, constraints, and operational characteristics of the system.

- **N.F.R.1: Performance (Latency):** A single multimodal analysis request (from input submission to result display) should be completed within an acceptable time frame, defined as less than 5 seconds to ensure a good user experience.
- **N.F.R.2: Accuracy:** The system's primary goal is to achieve a high classification accuracy. It must outperform standard unimodal (text-only) baseline models on a designated test dataset (e.g., the test split of the MVSA or Twitter dataset).
- **N.F.R.3: Modularity:** The system's design must be highly modular. Each component (text processing, image processing, fusion) must be loosely coupled, allowing for individual components to be upgraded or replaced (e.g., swapping BERT for RoBERTa) without requiring a redesign of the entire system.
- **N.F.R.4: Scalability:** While the prototype is for research, its architecture should not preclude future scalability. For instance, the feature extraction and classification modules should be designed as services that could, in theory, be scaled independently.
- **N.F.R.5: Usability (UI/UX):** The user interface must be simple, intuitive, and require minimal training. The user should be able to easily provide text and image inputs and clearly understand the output.
- **N.F.R.6: Maintainability:** The code must be well-documented, adhering to standard coding practices (e.g., PEP 8 for Python). All models, algorithms, and data pipelines must be clearly explained.
- **N.F.R.7: Portability:** The system should be developed using technologies (e.g., Python, TensorFlow/PyTorch, JavaScript) that are platform-independent and can be deployed on standard web-hosting environments or cloud platforms (AWS, GCP, Azure).

4.2 System Architecture

To meet the modularity and maintainability requirements (N.F.R.3, N.F.R.6), the system will be designed using a classic **N-Tier Layered Architecture**. This architecture separates concerns into logical layers, where each layer only communicates with the layers immediately adjacent to it.

Our system will be composed of four primary layers:

1. **Presentation Layer (Frontend):**
    - **Responsibility:** All user-facing interactions. This is the "User Interface" (UI).
    - **Technology:** Implemented as a web-based application using HTML, CSS, and JavaScript (possibly with a framework like React or Vue.js).
    - **Function:** Renders the input forms (text area, image upload), sends the input data to the Application Layer via an API request, and receives and displays the final classification result.
2. **Application Layer (Business Logic):**
    - **Responsibility:** Orchestrates the entire analysis process. It acts as the "brains" of the operation.
    - **Technology:** A backend server application (e.g., built with Python using a framework like Flask or FastAPI).
    - **Function:** Exposes a public API (e.g., /analyze) that the Presentation Layer calls. When it receives a request, it coordinates the workflow: it sends the text to the Text Processing Module, sends the image to the Image Processing Module, receives the feature vectors, passes them to the Fusion Module, and finally gets a prediction from the Classifier Module. It then packages this prediction and sends it back to the Presentation Layer.
3. **Service/Domain Layer (Core Modules):**
    - **Responsibility:** Contains the core, specialized AI/ML components of the project. This is where the novel research is implemented.
    - **Technology:** A collection of Python scripts and libraries (TensorFlow, PyTorch, Transformers, PIL).
    - **Function:** This layer is broken into the key modules defined in our project:
        - TextProcessingService: Loads the pre-trained BERT model, tokenizes raw text, and returns a feature vector.
        - ImageProcessingService: Loads the pre-trained ResNet model, pre-processes the image, and returns a feature vector.
        - FusionService: Implements the fusion algorithm (e.g., concatenation, attention, tensor fusion) to combine the two vectors.
        - ClassificationService: Contains the trained sentiment classifier (e.g., a simple Feed-Forward Network) that takes the fused vector and outputs a sentiment label.
4. **Data Layer (Persistence):**
    - **Responsibility:** Manages all persistent data, including the AI models themselves and any user data.
    - **Technology:**
        - **File System / Model Storage:** For storing the large, pre-trained model files (e.g., BERT's model.bin, ResNet's .h5 file).
        - **Database (Optional):** A SQL (e.g., PostgreSQL) or NoSQL (e.g., MongoDB) database to store user-submitted data, analysis results, and user accounts, satisfying F.R.9.

This layered design ensures a clean separation of concerns. The UI (Layer 1) doesn't need to know *how* a BERT model works. The Application Layer (Layer 2) doesn't need to know *how* to fuse vectors; it just needs to know *which* service to call. And the AI models (Layer 3) are completely independent of any user interface, making them reusable and easy to test.

4.3 Data Flow Diagram (DFD)

Data Flow Diagrams (DFDs) are used to visually represent the flow of data through a system. We will define the system at three levels of abstraction.

Level 0: Context Diagram

The Level 0 DFD (or Context Diagram) shows the entire system as a single process, highlighting its interaction with external entities.

- **External Entities:**
  - User: The person interacting with the system.
  - Model_Storage: An external data store holding the pre-trained AI models.
  - Result_Database: An external data store for logging analysis results.
- **Process:**
  - 0.0 Multimodal Sentiment Analysis System
- **Data Flows:**
  - User provides Text_Input and Image_Input to the System.
  - System provides Sentiment_Output to the User.
  - System retrieves Text_Model and Image_Model from Model_Storage.
  - System writes Analysis_Log to Result_Database.

Level 1: High-Level DFD

The Level 1 DFD breaks down the 0.0 process into its main sub-processes.

- **Processes:**
  - 1.0 Ingest Data
  - 2.0 Process Text
  - 3.0 Process Image
  - 4.0 Fuse Features
  - 5.0 Classify Sentiment
  - 6.0 Present Results
- **Data Stores:**

    - D1: Model_Storage
    - D2: Result_Database
- **Data Flows (Simplified):**
    1. User sends Raw_Input_Data (text + image) to 1.0 Ingest Data.
    2. 1.0 splits this into Raw_Text (sent to 2.0) and Raw_Image (sent to 3.0).
    3. 2.0 Process Text retrieves Text_Model from D1, processes Raw_Text, and produces Text_Vector.
    4. 3.0 Process Image retrieves Image_Model from D1, processes Raw_Image, and produces Image_Vector.
    5. Text_Vector and Image_Vector are sent to 4.0 Fuse Features, which produces a Fused_Vector.
    6. Fused_Vector is sent to 5.0 Classify Sentiment (which may also retrieve a Classifier_Model from D1).
    7. 5.0 produces Sentiment_Prediction, which is sent to 6.0 Present Results.
    8. Sentiment_Prediction is also logged as Analysis_Log to D2.
    9. 6.0 Present Results formats the Sentiment_Prediction into Sentiment_Output for the User.

Level 2: DFD (Example for Process 2.0)

The Level 2 DFD provides a granular, "zoomed-in" view of a single Level 1 process. Let's decompose 2.0 Process Text.

- **Processes (within 2.0):**
    - 2.1 Tokenize Text
    - 2.2 Load BERT Model
    - 2.3 Generate Embeddings
    - 2.4 Aggregate Vector
- **Data Stores:**
    - D1: Model_Storage
- **Data Flows (Internal):**
    1. Raw_Text arrives from 1.0.
    2. Raw_Text is sent to 2.1 Tokenize Text, which uses the BERT tokenizer to produce Token_IDs.
    3. 2.2 Load BERT Model retrieves the Text_Model from D1.
    4. Token_IDs and the Text_Model are sent to 2.3 Generate Embeddings, which produces Full_Token_Embeddings (a matrix of vectors, one for each token).
    5. Full_Token_Embeddings are sent to 2.4 Aggregate Vector. This process takes the embedding of the special [CLS] token (or averages all token vectors) to produce the final, single Text_Vector.
    6. Text_Vector is then sent out to process 4.0 Fuse Features.

4.4 Use Case Diagram and Description

The Use Case Diagram describes the functional interactions between actors (users) and the system.

- **Actors:**
  - User: The primary actor, representing a general user of the system (e.g., a researcher, a social media user).
  - Administrator: A privileged user responsible for maintaining the system.
- **Use Cases for User:**
  - **UC-1: Analyze Sentiment of Post:**
    - **Description:** The user provides a piece of text and an image to the system. The system performs the full multimodal analysis and returns a single sentiment label.
    - **Flow:**
      1. User accesses the main analysis page.
      2. User enters text into the text-input field.
      3. User uploads an image using the image-upload field.
      4. User clicks the "Analyze" button.
      5. System validates the input (text not empty, image is a valid format).
      6. System performs the analysis (processes 2.0, 3.0, 4.0, 5.0).
      7. System displays the result (process 6.0).
  - **UC-2: Analyze Text-Only Sentiment:**
    - **Description:** A variation of UC-1 where the user only provides text. The system detects the missing image, skips the image processing and fusion steps, and runs a text-only sentiment classifier.
  - **UC-3: Analyze Image-Only Sentiment:**
    - **Description:** A variation of UC-1 where the user only provides an image. The system detects the missing text, skips text processing and fusion, and runs an image-only sentiment classifier.
  - **UC-4: View Analysis History:**
    - **Description:** The user views a paginated list of their previous analysis requests, including the inputs and the system's results (F.R.9).
- **Use Cases for Administrator:**
  - **UC-5: Manage Models:**
    - **Description:** The administrator updates or replaces the core AI models (e.g., uploads a new, fine-tuned BERT model or a new fusion classifier) to improve system accuracy.
  - **UC-6: View System Logs:**
    - **Description:** The administrator views system-level logs to diagnose errors or monitor performance (N.F.R.1).

4.5 Class Diagram / ER Diagram

To satisfy F.R.9 (Analysis History), a database is required. An Entity-Relationship (ER) Diagram describes the structure of this database.

- **Entities (Tables):**
  - **User**
    - user_id (Primary Key, e.g., UUID or auto-incrementing integer)
    - username (string, unique)
    - password_hash (string)
    - created_at (timestamp)
  - **AnalysisRequest**
    - request_id (Primary Key, e.g., UUID)
    - user_id (Foreign Key, links to User.user_id)
    - input_text (text, nullable)
    - input_image_url (string, nullable)
    - request_timestamp (timestamp)
  - **AnalysisResult**

        - result_id (Primary Key, e.g., UUID)
        - request_id (Foreign Key, One-to-One with AnalysisRequest.request_id)
        - sentiment_label (string, e.g., "Positive", "Negative")
        - confidence_score (float)
        - text_vector (array/blob, optional for research)
        - image_vector (array/blob, optional for research)
        - fused_vector (array/blob, optional for research)
        - processing_time_ms (integer)
- **Relationships:**
    - **User to AnalysisRequest:** One-to-Many. One User can make many AnalysisRequests. Each AnalysisRequest belongs to exactly one User.
    - **AnalysisRequest to AnalysisResult:** One-to-One. Each AnalysisRequest has exactly one AnalysisResult. Each AnalysisResult corresponds to exactly one AnalysisRequest.

This structure allows the system to log every analysis, link it to a user, and store the inputs and outputs, which is invaluable for debugging, evaluating, and retraining the models.

4.6 Flowcharts and Algorithm Outline

This section outlines the core logic of the system as a flowchart.

Main Analysis Flowchart (Orchestrated by Application Layer)

1. **Start**
2. Receive HTTP POST request at /analyze endpoint.
3. Deserialize request data: raw_text and raw_image_file.
4. **Decision:** Is raw_text present and not empty?
    - **Yes:** Send raw_text to TextProcessingService. Receive text_vector.
    - **No:** Set text_vector to null.
5. **Decision:** Is raw_image_file present and valid?
    - **Yes:** Send raw_image_file to ImageProcessingService. Receive image_vector.
    - **No:** Set image_vector to null.
6. **Decision (Determine Analysis Type):**
    - **If text_vector is not null AND image_vector is not null:**
        - Proceed to "Multimodal Analysis" sub-flow (Step 7).
    - **If text_vector is not null AND image_vector is null:**
        - Proceed to "Text-Only Analysis" sub-flow (UC-2).
    - **If text_vector is null AND image_vector is not null:**
        - Proceed to "Image-Only Analysis" sub-flow (UC-3).
    - **If text_vector is null AND image_vector is null:**
        - Return Error: No input provided. **End**.
7. **Sub-Flow (Multimodal Analysis):**
    - Send text_vector and image_vector to FusionService.
    - Receive fused_vector.
    - Send fused_vector to ClassificationService.
    - Receive sentiment_prediction.
8. Log the analysis details to the Result_Database.
9. Format sentiment_prediction into a JSON response.
10. Send HTTP 200 (OK) response to Presentation Layer.
11. **End**

This logic ensures the system is robust and can handle all combinations of input as defined in the use cases.

4.7 Module-wise explanation

This section provides a detailed explanation of the implementation of each core module from the **Service Layer** (Layer 3).

4.7.1 Text Pre-processing and Feature Extraction Module

- **Purpose:** To convert a raw text string into a fixed-size, semantically rich feature vector.
- **Technology:** Transformers library (by Hugging Face), PyTorch or TensorFlow, and a pre-trained BERT-base-uncased model.
- **Workflow:**
  1. **Load Model:** The module loads the pre-trained BERT model and its corresponding WordPiece tokenizer into memory upon system startup.
  2. **Tokenization:** Receives a raw text string. The tokenizer is called to:
     - Convert text to lowercase.
     - Perform WordPiece tokenization.
     - Add special tokens: [CLS] (Classifier token) at the beginning and [SEP] (Separator token) at the end.
     - Pad or truncate the sequence to a fixed length (e.g., 128 tokens).
     - Generate an attention_mask (to tell the model to ignore padding tokens).
  3. **Inference:** The tokenized input_ids and attention_mask are passed to the BERT model in no_grad() or inference_mode() (to disable gradient calculations, saving memory and computation).
  4. **Embedding Generation:** The model returns its full output. We extract the last_hidden_state.
  5. **Aggregation:** To get a single vector for the entire sentence, we select the hidden state corresponding to the first token, [CLS]. This [CLS] token's final embedding is designed to be a summary representation of the entire sequence, making it ideal for classification.
  6. **Output:** The module returns this 768-dimension [CLS] vector as the final text_vector.

4.7.2 Image Feature Extraction Module

- **Purpose:** To convert a raw image file into a fixed-size, content-rich feature vector.
- **Technology:** Pillow (PIL) for image processing, PyTorch or TensorFlow, and a pre-trained ResNet-50 model (trained on ImageNet).
- **Workflow:**
  1. **Load Model:** The module loads the pre-trained ResNet-50 model, with its final classification layer (the "head") removed.
  2. **Image Pre-processing:** Receives a raw image file. It is processed through a standard ImageNet transformation pipeline:
     - Open the image file.
     - Resize it to a short-side length of 256 pixels.
     - Take a 224x224 center crop.
     - Convert the image to a PyTorch/TF tensor.

    - Normalize the pixel values using the ImageNet mean and standard deviation.
3. **Inference:** The processed image tensor is passed to the "headless" ResNet-50 model (also in inference mode).
4. **Feature Generation:** The model's output will be the activations from its final pooling layer.
5. **Output:** This process results in a 2048-dimension vector. This image_vector represents the high-level visual features (e.g., "contains a dog," "is a beach scene," "has a smiling face") of the image.

4.7.3 Multimodal Fusion Module

- **Purpose:** To combine the text_vector and image_vector into a single, unified representation. This is the core of the multimodal research.
- **Technology:** PyTorch or TensorFlow.
- **Workflow (Baseline Implementation - Concatenation):**
    1. **Input:** Receives the text_vector (768 dimensions) and image_vector (2048 dimensions).
    2. **Projection (Optional but recommended):** The vectors are of different sizes and come from different "spaces." A simple concatenation may not be optimal. It's common to first project them to a common dimension (e.g., 512) using a small, trainable linear layer (a.k.a. a dense layer or fully connected layer) for each.
        - text_vector_proj = Linear_Text(text_vector) (output: 512-dim)
        - image_vector_proj = Linear_Image(image_vector) (output: 512-dim)
    3. **Fusion:** The two projected vectors are concatenated.
        - fused_vector = concatenate([text_vector_proj, image_vector_proj])
    4. **Output:** The module returns the fused_vector (now 1024 dimensions).
- **Note:** This module is designed to be replaceable. More advanced fusion techniques (e.g., cross-attention, tensor fusion) would be implemented as alternative "strategies" within this module, adhering to N.F.R.3.

4.7.4 Sentiment Classifier Module

- **Purpose:** To take the final fused_vector and produce a human-readable sentiment label.
- **Technology:** PyTorch or TensorFlow.
- **Workflow:**
    1. **Architecture:** This module contains a simple Feed-Forward Network (FFN), also known as a Multi-Layer Perceptron (MLP). This network is *not* pre-trained; it will be trained from scratch on our multimodal dataset.
    2. **Input:** Takes the fused_vector (e.g., 1024 dimensions).
    3. **Structure (Example):**
        - Input Layer (1024 nodes)
        - Hidden Layer 1 (512 nodes) + ReLU Activation + Dropout (0.4)
        - Hidden Layer 2 (256 nodes) + ReLU Activation + Dropout (0.4)
        - Output Layer (N nodes) + Softmax Activation
        - The number of output nodes N depends on the classification task (e.g., N=3 for

Positive/Negative/Neutral, or N=4 to include Sarcasm).

4. **Training:** This FFN (along with the optional projection layers from 4.7.3) is the *only* part of the model that is trained on our specific sentiment dataset. The BERT and ResNet "backbones" remain frozen to leverage their pre-trained knowledge.
5. **Output:** During inference, the module takes the fused_vector, passes it through the FFN, and performs an argmax on the final Softmax output to get the predicted class label (e.g., [0.1, 0.8, 0.1] $\rightarrow$ Class 1 $\rightarrow$ "Positive").

4.7.5 Frontend/UI Module

- **Purpose:** To provide a simple, usable web interface for the system (N.F.R.5).
- **Technology:** HTML5, CSS3 (e.g., TailwindCSS), and JavaScript.
- **Workflow:**
  1. **Render:** A single-page application (SPA) is loaded in the user's browser. It displays a title, a text area, and an "Upload Image" button.
  2. **Input Handling (JavaScript):**
     - When an image is selected, it is stored in the browser's memory. A preview is shown.
     - When the "Analyze" button is clicked, an event listener fires.
  3. **API Call:** The JavaScript constructs a FormData object.
     - It appends the text: formData.append('text', textArea.value).
     - It appends the image: formData.append('image', imageFile).
  4. **Fetch:** It uses the fetch() API to send this FormData as a POST request to the backend's /analyze endpoint (the Application Layer). A loading spinner is shown.
  5. **Response Handling:**
     - The frontend awaits the JSON response from the backend.
     - When the response is received, the loading spinner is hidden.
     - The sentiment_label and confidence_score are extracted from the JSON.
     - The result is displayed in a designated "Results" div on the page.

Chapter 5: Dataset and Preprocessing

Goal: Document your dataset and data cleaning process.

Following the system's architectural design in Chapter 4, this chapter addresses the most critical raw material for any machine learning project: the data. The performance, generalizability, and ultimate success of our Multimodal Sentiment Analysis (MSA) system are fundamentally dependent on the quality and characteristics of the dataset used for training and evaluation.

This chapter provides a comprehensive overview of the dataset selected for this research. We will describe its origin, structure, and statistical properties. We will then meticulously detail the extensive data cleaning and preprocessing pipelines required to transform raw, "in-the-wild" data into a clean, structured, and model-ready format. This includes separate, specialized pipelines for both text and image modalities, as well as a strategy for handling the common problem of class imbalance. Finally, we will present an Exploratory Data Analysis (EDA) to uncover patterns and justify our preprocessing choices, complete with visual examples that highlight the challenges this dataset presents.

5.1 Dataset Description

After a thorough review of available resources, the **MVSA (Multimodal-Sentiment-Analysis)** dataset, as introduced by Niu, et al. (2016), has been selected for this research. This dataset is a standard and widely-used benchmark in the field of multimodal sentiment analysis, making it ideal for evaluating our model's performance against existing literature.

The dataset is sourced from **Twitter**, which is of particular interest to this research. Twitter data is inherently multimodal, as users frequently combine short, informal text (tweets) with images. This data is also notoriously "noisy," containing a high volume of slang, abbreviations, hashtags, and emojis, which presents a realistic and challenging test bed for our NLP module.

The MVSA dataset is composed of two primary subsets, which are both crucial for our

research goals:

1. **MVSA-Single:** This subset contains text-image pairs where the sentiment label is assigned to the *entire post* as a single unit. The underlying assumption is that the text and image are complementary and jointly express a single, unified sentiment (e.g., a photo of a sunset with the text "What a beautiful evening!").
2. **MVSA-Multiple:** This subset is more complex and directly addresses the research gap identified in Chapter 2. In this subset, the text and the image were labeled *independently*. This allows for the identification of **incongruent** pairs, where the sentiment of the text may differ from the sentiment of the image (e.g., a smiling photo with sarcastic text). This subset is essential for training and evaluating our model's ability to handle sarcasm and semantic dissonance.

For the purpose of this dissertation, we will combine these subsets to create a robust dataset that features both complementary and incongruent data points, thereby training a model that is sensitive to the complex interplay between modalities.

5.2 Dataset Size, Structure, and Attributes

The combined MVSA dataset contains approximately **5,129** high-quality, manually-annotated text-image pairs. While this is not "big data" by modern standards, its high quality and manual labeling make it an extremely valuable resource. The limited size also strongly reinforces the design choice from Chapter 4 to use **transfer learning**, as training deep learning models from scratch on 5,000 samples would be infeasible.

Data Structure

The dataset is provided as a collection of image files and a metadata file (e.g., .csv or .json) that links the images to their corresponding text and labels. Each record in the metadata file represents a single multimodal post and contains the following attributes:

- **image_id:** A unique identifier (e.g., 1001.jpg) that corresponds to an image file in the dataset directory.
- **text_content:** A string containing the raw text of the tweet associated with the image.
- **sentiment_label:** The ground-truth sentiment label for the post.
- **label_type:** An attribute that may specify if the sample is from MVSA-Single or MVSA-Multiple.

Attributes and Labels

The primary attribute for our classification task is the sentiment_label. The dataset is annotated with three distinct sentiment classes:

1. **Positive:** The post expresses a clearly positive emotion, opinion, or state (e.g., joy, love,

excitement).
2. **Negative:** The post expresses a clearly negative emotion, opinion, or state (e.g., sadness, anger, disgust).
3. **Neutral:** The post is primarily objective, factual, or does not convey any strong sentiment (e.g., a simple observation, a news-like statement).

A preliminary analysis of the class distribution reveals a moderate class imbalance, which will be discussed and addressed in Section 5.6.

5.3 Data Cleaning and Labeling

While the dataset is pre-labeled, the raw text content is sourced directly from Twitter and requires significant cleaning before it can be used for any meaningful feature extraction. The image data also requires validation.

Labeling Process

We are using the annotations provided by the dataset's creators. It is important to note their methodology to trust the quality of our ground truth. The labels were assigned by multiple human annotators, and a high **inter-annotator agreement (IAA)** score was reported, indicating that the labels are consistent and reliable. For the MVSA-Multiple subset, annotators were specifically instructed to label the text and image separately, providing the granular labels needed to identify incongruity.

Text Data Cleaning

The text cleaning pipeline is designed to remove "noise" that provides no semantic value, while carefully preserving tokens that are rich in sentiment. This is a multi-step process:

1. **Lowercase Conversion:** All text is converted to lowercase to ensure uniformity (e.g., "Love" and "love" are treated as the same word).
2. **URL Removal:** All hyperlinks (e.g., http://t.co/..., https://...) are identified via regular expressions and removed. These are artifacts of the Twitter platform and carry no direct sentimental content.
3. **Username/@Mention Removal:** All Twitter usernames (@username) are removed. These act as pointers to other users and are considered noise for sentiment classification.
4. **Hashtag (#) Handling:** Hashtags are a nuanced challenge. Simply removing them (#Happy) would discard valuable data. Instead, we "unpack" them: the # symbol is removed, and the hashtag content is split into words (e.g., #HappyDay becomes "Happy Day"). This preserves the sentiment-rich words.
5. **Emoji Conversion:** Emojis are a critical source of sentiment. Instead of removing them, we **convert them to their textual representation**. For example, 😊 is converted to

":smiling_face:". This allows the NLP model to learn the semantic meaning of the emoji just like any other word.

6. **Special Character Removal:** Non-ASCII characters, HTML entities (&, <), and miscellaneous non-alphanumeric symbols (e.g., *, ^, ~) are removed.
7. **Punctuation Handling:** Most punctuation is removed, but sentiment-carrying punctuation like exclamation marks (!) and question marks (?) are preserved, as they can modify the intensity of a statement.

Image Data Cleaning

1. **Corrupted File Check:** A script is run to check all image files in the dataset. Any 0-byte, corrupted, or unopenable files are identified and their corresponding entries are removed from the metadata file.
2. **Deduplication:** To prevent data leakage (having the same image in both the training and testing sets), we perform visual deduplication. We use a perceptual hash (pHash) algorithm to generate a "fingerprint" for each image. Images with identical or near-identical hashes are identified, and duplicates are removed.

5.4 Text Preprocessing Pipeline (Model-Specific)

After the text is *cleaned* (Section 5.3), it must be *preprocessed* into the specific numerical format expected by our chosen NLP model, **BERT** (as defined in Chapter 4). This is a separate and distinct pipeline.

1. **Tokenization:** The cleaned text string is passed to a pre-trained BERT-base-uncased WordPiece tokenizer. This tokenizer splits the text into subword tokens, handling out-of-vocabulary words gracefully (e.g., "multimodal" might become ["multi", "##modal"]).
2. **Adding Special Tokens:** The tokenizer automatically adds the required special tokens for a classification task:
    - [CLS] (Classifier Token): Prepended to the beginning of the sequence. The final embedding of this token will be used as the aggregate representation of the entire

text.
    - [SEP] (Separator Token): Appended to the end of the sequence.
3. **Padding and Truncation:** To ensure all inputs are a uniform length for batch processing, we set a MAX_LENGTH. Based on an analysis of tweet lengths (see Section 5.7), a MAX_LENGTH of 64 tokens is chosen.
    - Sequences *shorter* than 64 tokens are **padded** with [PAD] tokens.
    - Sequences *longer* than 64 tokens are **truncated**.
4. **Attention Mask Generation:** An attention_mask (a tensor of 1s and 0s) is created. This mask tells the BERT model which tokens are "real" (value 1) and which are padding (value 0), ensuring the model does not "attend" to the padding.
5. **Output Tensors:** The final output of this pipeline is a set of PyTorch/TensorFlow tensors: input_ids, attention_mask, and token_type_ids (if needed), all of an identical shape (e.g., batch_size, 64).

5.5 Image Preprocessing Pipeline (Model-Specific)

Similarly, the cleaned images must be preprocessed into the exact format expected by our chosen vision model, **ResNet-50**.

1. **Image Loading:** The image file is loaded from disk using the Pillow (PIL) library and ensured to be in RGB format (discarding any alpha/transparency channels).
2. **Transformation Pipeline:** The image is passed through a transforms.Compose pipeline, which is standard practice for models pre-trained on ImageNet.
    - **Resize:** The image is first resized to 256x256 pixels.
    - **CenterCrop:** A 224x224 pixel crop is taken from the center of the image. This is the mandatory input size for the ResNet-50 model.
    - **ToTensor:** The PIL image (with pixel values from 0-255) is converted into a PyTorch/TF tensor (with values from 0.0-1.0).
    - **Normalize:** The tensor is normalized using the standard ImageNet mean ([0.485, 0.456, 0.406]) and std ([0.229, 0.224, 0.225]). This is a critical step, as the model was trained on images normalized in this exact way.

The output of this pipeline is a 3-channel tensor of shape (3, 224, 224) ready to be fed into the

ResNet-50 feature extractor.

5.6 Handling Class Imbalance

Exploratory Data Analysis (Section 5.7) reveals that the dataset is not perfectly balanced. The Positive class is the most frequent, followed by Negative and Neutral.

- **Problem:** If trained on this imbalanced data, our model will develop a "bias" towards the majority (Positive) class. It will achieve high accuracy by simply guessing "Positive" most of the time, while performing very poorly on the under-represented Neutral and Negative classes. This would result in a high "accuracy" but a very low and "useless" **F1-Score**.
- **Solutions Considered:**
  1. **Oversampling (e.g., SMOTE):** Synthetically creating new data points for the minority classes. This was rejected due to the risk of overfitting and the complexity of synthetically generating *multimodal* data.
  2. **Undersampling:** Randomly removing data points from the majority class. This was rejected because our dataset is already small, and discarding valuable data is not an acceptable option.
- Chosen Solution: Weighted Loss Function:
  This is the most effective and non-intrusive solution. We will modify our training process at the loss function level (e.g., CrossEntropyLoss).
  1. We will calculate a "class weight" for each class, which is the inverse of its frequency in the training data.
  2. **Example:**
     - Positive: 60% of data $\rightarrow$ weight = 1 / 0.60 = 1.67
     - Negative: 25% of data $\rightarrow$ weight = 1 / 0.25 = 4.00
     - Neutral: 15% of data $\rightarrow$ weight = 1 / 0.15 = 6.67
  3. These weights (normalized) are passed to our loss function. When the model makes a mistake on a Neutral sample, the resulting loss will be multiplied by 6.67, while a mistake on a Positive sample is only multiplied by 1.67. This "forces" the model to pay significantly more attention to the minority classes, leading to a more balanced and useful classifier.

5.7 Exploratory Data Analysis (EDA)

EDA was performed to understand the dataset's characteristics and to validate the design choices made above.

Text EDA

- **Tweet Length Distribution:** A histogram of token counts (after cleaning) was plotted. The vast majority of tweets (95%) fall under 60 tokens. This strongly supports our choice of a MAX_LENGTH of 64 (from Section 5.4), as it captures nearly all the data without unnecessary computational overhead from padding.
- N-Gram Analysis: We analyzed the most common 1-grams (words) and 2-grams (bigrams) after stop-word removal for each sentiment class.
  *
  - **Positive Class:** Dominated by words like "love," "happy," "beautiful," "amazing," and "good morning."
  - **Negative Class:** Dominated by words like "hate," "sad," "miss," "bad," and "can't."
  - **Neutral Class:** Dominated by more objective words, location names, and brand names (e.g., "new," "phone," "today," "London").
  - This analysis confirms that the text content is a strong (but not a complete) indicator of sentiment.

Image EDA

- **Image Dimensions:** An analysis of the raw image files showed a wide variety of aspect ratios and sizes, from 1:1 square "Instagram-style" photos to wide 16:9 panoramic shots. This confirms that a resize-and-crop strategy (Section 5.5) is not just a best practice, but a necessity.
- **Visual Content (Qualitative):** A qualitative review of images shows a strong correlation for simple cases. Positive images frequently contain smiles, bright colors, and scenic landscapes. Negative images often contain dark scenes, "fail" or "accident" themes, or sad facial expressions.

Cross-Modal EDA

- **Sentiment Co-occurrence (for MVSA-Multiple):** We created a confusion matrix to visualize the relationship between the *text-only sentiment* and the *image-only sentiment*. This analysis revealed that while the majority of samples are **complementary** (e.g., Positive-Text + Positive-Image), a significant portion (approx. 15-20%) are **incongruent** (e.g., Positive-Text + Negative-Image or vice-versa). This subset quantifies our core research problem and will be used to create a specialized "sarcasm/incongruity" test set.

5.8 Visualization of Samples

To ground this discussion, the following are representative samples from the dataset that illustrate the range of challenges.

Example 1: Clear Positive (Complementary)

- **Image:** A person smiling on a scenic beach.
- **Text:** "Having the most amazing time on my vacation! ☀️😊 #blessed"
- **Label:** Positive
- **Analysis:** A straightforward case. Both text-only and image-only models would likely classify this as Positive.

Example 2: Clear Negative (Complementary)

- **Image:** A car with a flat tire on the side of a road.
- **Text:** "My day is completely ruined... stuck in the middle of nowhere. 😠"
- **Label:** Negative
- **Analysis:** Another complementary case. The text ("ruined") and image ("flat tire") strongly agree.

Example 3: Sarcasm/Incongruity (The Core Challenge)

- **Image:** A person at an office desk, smiling, but surrounded by a huge pile of work.
- **Text:** "Just got *another* urgent request at 8 PM. I just *love* my job so much."
- **Label:** Negative
- **Analysis:** This is the critical failure case for unimodal models.
  - **Text-Only Model:** Sees "love my job so much" $\rightarrow$ Positive.
  - **Image-Only Model:** Sees "smiling person" $\rightarrow$ Positive.
  - **Multimodal Model:** Must learn that the *combination* of "love my job" and an image of "excessive work" (the pile of paper) is a strong indicator of sarcasm, and the true sentiment is Negative. This sample perfectly illustrates the necessity of a sophisticated fusion module.

Example 4: Neutral (Objective)

- **Image:** A standard product shot of a new laptop on a clean desk.
- **Text:** "Got the new XPS 15. The screen is 16:10."
- **Label:** Neutral
- **Analysis:** This post is factual and objective. It doesn't express strong emotion. It's important to train the model to recognize and output Neutral for such cases, rather than forcing a Positive or Negative label.

This chapter has established a solid foundation by selecting a high-quality dataset, defining rigorous and separate pipelines for cleaning and preprocessing, and anticipating real-world challenges like class imbalance. The insights from our EDA and the concrete examples from the data confirm the complexity of the task and validate the design of our system. The data is now prepared for the model training and implementation phase described in Chapter 6.

Chapter 6: Model Implementation

Goal: Explain your full implementation process step-by-step.

This chapter bridges the gap between design and execution. Building upon the theoretical foundations in Chapter 3, the system architecture in Chapter 4, and the prepared dataset from Chapter 5, this chapter details the "build" phase of the project. It provides a comprehensive, step-by-step account of the software, tools, and code used to construct the proposed Multimodal Sentiment Analysis (MSA) system.

We will cover the complete implementation process, from establishing the development environment to the granular code architecture of each component. This includes the implementation of the separate text and image feature extraction backbones, the design of the novel fusion module, the methodology for training and validation, and the optimization techniques used to achieve robust and accurate results. This chapter is intended to serve as a technical blueprint, ensuring the work is transparent, reproducible, and clearly-structured.

6.1 Tools and Libraries Used

The selection of tools is critical for building a modern, efficient, and reproducible machine learning system. The following open-source technologies were chosen for their robust performance, extensive documentation, and widespread adoption in the research community.

- **Primary Language: Python (Version 3.10+)**
  - The *lingua franca* of data science and machine learning. Its rich ecosystem of libraries makes it the only logical choice for this project.
- **Core ML Framework: PyTorch (Version 2.0+)**
  - Chosen over TensorFlow for its "Pythonic" feel, dynamic computation graph (Eager Mode), and strong preference in the research community, which provides a wealth of pre-trained models and tutorials.
- **NLP Toolkit: Hugging Face transformers**
  - The de facto standard for all Transformer-based models. This library provides a

simple, high-level API to download, load, and use pre-trained models like BERT, along with their corresponding tokenizers.

- **Computer Vision Toolkit: torchvision and Pillow (PIL)**
  - torchvision provides access to pre-trained vision models like ResNet-50 and a suite of high-performance image transformation tools (transforms.Compose).
  - Pillow (PIL) is used for the practical task of opening, reading, and converting image files from disk.
- **Data Handling: pandas, NumPy, scikit-learn**
  - pandas: Used to load and manipulate the dataset's .csv metadata file.
  - NumPy: The fundamental package for numerical operations, forming the backbone for all tensor manipulations.
  - scikit-learn: Used for supplementary tasks: splitting the dataset (train_test_split) and calculating evaluation metrics (e.g., accuracy_score, precision, recall, f1_score).
- **API & Deployment: FastAPI and uvicorn**
  - FastAPI was chosen for the Application Layer (Chapter 4) to serve the final model. It is a modern, high-performance web framework that is asynchronous by default and includes automatic interactive documentation.
  - uvicorn: The ASGI server used to run the FastAPI application.
- **Plotting: matplotlib and seaborn**
  - Used during the Exploratory Data Analysis (Chapter 5) and for generating final results, such as loss curves and confusion matrices.

6.2 Environment Setup and Architecture

The development and deployment environments are standardized to ensure consistency and performance.

- **Software Architecture:**
  - **Operating System:** All development and training are performed on a Linux (Ubuntu 22.04) environment to ensure compatibility with CUDA.
  - **Dependency Management:** A Python virtual environment (venv) is used to isolate project dependencies. All required libraries are frozen into a requirements.txt file for reproducibility.
  - **Key Dependencies:**
    - python == 3.10.x
    - torch == 2.0.x (with CUDA 11.8/12.1 support)
    - transformers == 4.3x.x
    - scikit-learn == 1.3.x
    - pandas == 2.0.x
    - fastapi == 0.103.x
- **Hardware Architecture:**

- **Training & Development:** Due to the computational cost of the large backbone models (BERT and ResNet-50), training is not feasible on consumer-grade hardware. The model training and hyperparameter tuning phases are conducted on a cloud-based Virtual Machine (e.g., Google Cloud n1-standard-8) equipped with an **NVIDIA A100** or **V100** GPU with at least 16 GB of VRAM. This is essential for handling the models and batch processing.
- **Inference (Deployment):** The final, trained API is designed to be deployed on a more cost-effective cloud VM, such as an **NVIDIA T4** GPU instance. This provides a balance of performance and cost for real-time analysis requests.

6.3 Text Sentiment Model (BERT Feature Extractor)

As outlined in Chapter 4 (Module 4.7.1), we use a pre-trained BERT model as a "frozen" feature extractor. This implementation avoids fine-tuning the 110 million parameters of BERT, which saves enormous computational cost and leverages the model's vast, pre-trained knowledge of the English language.

- **Model Selection:** bert-base-uncased from the Hugging Face model hub.
- **Implementation:**
  1. **Loading:** The model (AutoModel) and its tokenizer (AutoTokenizer) are loaded from the transformers library.
  2. **Freezing:** This is the most critical step. We iterate through all parameters of the loaded BERT model and set their requires_grad attribute to False. This ensures that no gradients will be backpropagated into the BERT model during training; it will *only* be used for forward-pass inference.
  3. **Mode:** The model is permanently put into evaluation mode (model.eval()) to deactivate dropout layers.
- **Feature Extraction Function (get_text_vector):**
  - **Input:** A raw text string.
  - **Tokenization:** The text is passed through the pre-processing pipeline defined in Chapter 5. The AutoTokenizer is called with:
    - max_length=64 (as determined by our EDA)
    - padding='max_length'
    - truncation=True
    - return_tensors='pt' (to get PyTorch tensors)
  - **Inference:** The resulting input_ids and attention_mask tensors are passed to the frozen BERT model within a torch.no_grad() context to disable gradient calculation.
  - **Aggregation:** The model's last_hidden_state is retrieved. We select the output vector corresponding to the first [CLS] token (index [:, 0, :]).
  - **Output:** A single tensor of shape **(1, 768)**.

6.4 Image Sentiment Model (ResNet-50 Feature Extractor)

Parallel to the text model, the ResNet-50 model is implemented as a frozen visual feature extractor, as per Chapter 4 (Module 4.7.2).

- **Model Selection:** ResNet-50 (pre-trained on ImageNet) from torchvision.models.
- **Implementation:**
  1. **Loading:** The model is loaded using models.resnet50(weights=ResNet50_Weights.IMAGENET1K_V2).
  2. **"Headless" Mode:** The ResNet-50 model is pre-trained for 1000-class ImageNet classification. We do not need these 1000 classes. To convert it into a feature extractor, we "remove the head" (its final fully-connected layer) by replacing it with a simple torch.nn.Identity() layer. The output of the model will now be the 2048-dimension feature vector from the preceding global average pooling layer.
  3. **Freezing:** As with BERT, we iterate through all parameters and set requires_grad = False.
  4. **Mode:** The model is permanently set to model.eval().
- **Feature Extraction Function (get_image_vector):**
  - **Input:** A raw image file path.
  - **Preprocessing:** The image is loaded with Pillow, converted to RGB, and passed through the torchvision transformation pipeline defined in Chapter 5 (Resize to 256, CenterCrop to 224, ToTensor(), and Normalize()).
  - **Inference:** The resulting tensor is unsqueeze(0) to create a batch of one. It is then passed to the frozen, headless ResNet-50 model within a torch.no_grad() context.
  - **Output:** A single tensor of shape **(1, 2048)**.

6.5 Feature Extraction and Fusion Method

This is the implementation of the *trainable* component of our system (Modules 4.7.3 and 4.7.4). The "backbones" (BERT, ResNet) are frozen, but the "head" (our fusion/classifier network) is trained from scratch on the MVSA dataset.

To vastly accelerate training, we adopt a **two-stage process**:

1. **Stage 1: Pre-Extraction (Offline):** We run *every* sample in our train, val, and test sets through the get_text_vector and get_image_vector functions *once*. We save these (text_vector, image_vector, label) tuples to disk.
2. **Stage 2: Training (Online):** Our training script loads these *pre-extracted features* instead of the raw text/images. This means the 110M-parameter BERT and 23M-parameter ResNet models are *never* loaded during training, allowing us to train and iterate extremely quickly on our small, trainable fusion network.

Fusion Model (MultimodalClassifier(torch.nn.Module)):

This class defines our trainable network.

- **Architecture (implements 4.7.3 and 4.7.4):**
  1. **Text Projection Layer:** torch.nn.Linear(in_features=768, out_features=512)
  2. **Image Projection Layer:** torch.nn.Linear(in_features=2048, out_features=512)
  - *Design note: These projection layers are crucial. They reduce the dimensionality and map the features from two very different models (BERT and ResNet) into a common, 512-dimension "fusion space."*
  3. **Classifier Head (FFN):** A torch.nn.Sequential block:
     - torch.nn.ReLU()
     - torch.nn.Dropout(p=0.5)
     - torch.nn.Linear(in_features=1024, out_features=512) (1024 = 512 + 512 from concatenation)
     - torch.nn.ReLU()
     - torch.nn.Dropout(p=0.5)
     - torch.nn.Linear(in_features=512, out_features=3) (for Positive, Negative, Neutral)
- **forward(self, text_vec, img_vec) method:**
  1. text_proj = self.text_projection_layer(text_vec)
  2. img_proj = self.image_projection_layer(img_vec)
  3. fused_vector = torch.cat((text_proj, img_proj), dim=1)
  4. logits = self.classifier_head(fused_vector)
  5. return logits

6.6 Model Training and Validation Steps

The train.py script executes the training and validation logic.

1. **Dataset and DataLoaders:**
   - A custom PyTorch Dataset class (PreExtractedMVSADataset) is implemented. Its __init__ loads the pre-extracted feature files. Its __getitem__ simply returns the (text_vector, image_vector, label) for a given index.
   - The dataset is split into train_dataset and val_dataset (e.g., 80/20 split).
   - These are wrapped in torch.utils.data.DataLoader (e.g., batch_size=64, shuffle=True for the training loader).
2. **Model, Loss, and Optimizer Initialization:**
   - model = MultimodalClassifier().to(device)
   - loss_fn = ... (See Section 6.8)
   - optimizer = ... (See Section 6.8)
3. **Training Loop (for epoch in ...):**

- The model is set to model.train().
- Iterate through batches from the train_loader.
- For each batch (text_vecs, img_vecs, labels):
  a. Move tensors to the GPU (.to(device)).
  b. optimizer.zero_grad(): Clear previous gradients.
  c. logits = model(text_vecs, img_vecs): Perform a forward pass.
  d. loss = loss_fn(logits, labels): Calculate the batch loss.
  e.s. loss.backward(): Backpropagate gradients.
  f. optimizer.step(): Update model weights.
- The average training loss for the epoch is logged.

4. **Validation Loop:**
   - The model is set to model.eval() and enclosed in a torch.no_grad() block.
   - Iterate through batches from the val_loader.
   - Calculate val_loss and store all predictions and true_labels.
   - After the loop, compute metrics: val_accuracy and val_f1 (weighted F1-score, as it's our primary metric for an imbalanced dataset).
5. **Model Checkpointing:**
   - The val_f1 is compared against the best_val_f1 seen so far.
   - If val_f1 > best_val_f1, the current model's state dictionary is saved to disk: torch.save(model.state_dict(), 'best_model.pth'). This ensures we keep the best-performing model.

6.7 Hyperparameter Tuning

Before the final training run, a hyperparameter search is conducted to find the optimal settings for our fusion classifier. We use a simple **Grid Search** methodology, training for a limited number of epochs (e.g., 10) for each combination.

The following hyperparameters are tuned:

- **Learning Rate (for Adam):** [1e-3, 5e-4, 1e-4, 1e-5]
- **Batch Size:** [32, 64, 128]
- **Dropout Rate:** [0.3, 0.4, 0.5, 0.6]
- **Projection Dimension:** [256, 512, 768]
- **Classifier Hidden Layers:** [(512, 256), (512, 128), (256, 128)]

The combination yielding the highest val_f1 score is selected for the final, full training run.

6.8 Loss Functions and Optimization Methods

This section implements the critical choices made in Chapter 5 to handle our dataset's specific challenges.

- **Loss Function: Weighted Cross-Entropy Loss**
  - As defined in Chapter 5 (Section 5.6), we *must* use a weighted loss to combat class imbalance.
  - **Implementation:**
    1. First, we calculate the class frequencies from the training set (e.g., [0.6, 0.25, 0.15]).
    2. The weights are calculated as the inverse frequency: weights = 1.0 / torch.tensor([0.6, 0.25, 0.15]).
    3. The weights are normalized and moved to the GPU: class_weights = weights / weights.sum().
    4. The loss function is instantiated with these weights: loss_fn = torch.nn.CrossEntropyLoss(weight=class_weights.to(device)).
  - This "punishes" the model more for misclassifying rare (Negative, Neutral) samples, forcing it to learn their features.
- **Optimization Method: Adam with Scheduler**
  - **Optimizer:** torch.optim.Adam. It is the most robust and widely used default optimizer. It combines the benefits of momentum and adaptive learning rates.
  - **Learning Rate Scheduler:** We also implement torch.optim.lr_scheduler.ReduceLROnPlateau. This monitors the validation_loss. If the loss stops improving for a "patience" of N epochs (e.g., 2), the learning rate is automatically reduced (e.g., by a factor of 10). This helps the model settle into a fine-grained minimum.

6.9 Code Architecture and Module Structure

The project directory is organized for a clean separation of concerns, distinguishing between model R&D, data, and the final deployable application (as planned in Ch 4).

```
/Multimodal-Sentiment-Project/
|
|-- /app/                # Layer 2: FastAPI Application
|   |-- main.py            # API endpoints (/analyze)
|   |-- services.py        # Layer 3: Contains feature extraction & prediction functions
|   |-- models.py          # Pydantic request/response models
|
|-- /ml_core/            # Core ML code, non-deployable
```

```
|   |-- /checkpoints/
|   |   |-- best_model.pth    # Our trained fusion classifier (10-20MB)
|   |
|   |-- network.py          # Definition of MultimodalClassifier (nn.Module)
|   |-- dataset.py          # MVSADataset class
|   |-- train.py            # The main training script
|   |-- pre_extract.py      # Script to run Stage 1 (feature pre-extraction)
|
|-- /notebooks/
|   |-- 01_EDA_and_Cleaning.ipynb # (As described in Chapter 5)
|
|-- /data/
|   |-- /mvsa_dataset/
|   |   |-- images/         # All .jpg files
|   |   |-- metadata.csv    # CSV with text and labels
|   |-- /pre_extracted/
|   |   |-- train_features.pt # Saved output from pre_extract.py
|   |   |-- val_features.pt
|
|-- /assets/                # Pre-trained models (e.g., BERT, ResNet)
|   |-- /bert-base-uncased/   # (Cached by Hugging Face)
|   |-- /resnet50/          # (Cached by torchvision)
|
|-- requirements.txt        # All Python dependencies
|-- README.md
```

6.10 Flow of Execution

This section describes the two primary workflows of the system: (1) the one-time, offline training process and (2) the real-time, online inference process.

1. Training Flow (Offline)

This flow is executed once to create our best_model.pth file.

1. **Start:** Developer runs pre_extract.py.
2. The script loads the full MVSA dataset.
3. It loads the **frozen BERT** and **frozen ResNet-50** models.
4. It iterates through all 5,129 samples, generating and saving the (768-dim) and (2048-dim) vectors for each.
5. It saves the outputs to train_features.pt and val_features.pt.
6. **Next:** Developer runs train.py.

7. This script *only* loads the small, lightweight MultimodalClassifier.
8. It loads the pre_extracted features from disk.
9. It initializes the WeightedCrossEntropyLoss and Adam optimizer.
10. It begins the train/validate loop (Section 6.6), training *only* the fusion head.
11. **End:** The script saves the highest-performing fusion head as best_model.pth.

2. Inference Flow (Online)

This is the live "analysis" workflow, executed by the FastAPI server.

1. **Start:** User submits text and image to the frontend, which sends a POST request to the /analyze endpoint.
2. app/main.py receives the request.
3. It calls app/services.py -> get_prediction(text, image):
4. Inside services.py:
   a. text_vector = get_text_vector(text):
   i. Loads the frozen BERT model (this is slow on first load, but fast after).
   ii. Performs full tokenization and inference.
   iii. Returns (1, 768) vector.
   b. image_vector = get_image_vector(image):
   i. Loads the frozen ResNet-50 model.
   ii. Performs full preprocessing and inference.
   iii. Returns (1, 2048) vector.
   c. logits = get_fused_prediction(text_vector, image_vector):
   i. Loads the trained MultimodalClassifier (i.e., best_model.pth).
   ii. Performs the forward pass (projection, concatenation, classification).
   iii. Returns (1, 3) logit vector.
5. main.py applies a softmax to the logits to get probabilities, finds the argmax for the final label (e.g., "Positive").
6. **End:** The JSON response {"sentiment": "Positive", "confidence": 0.88} is returned to the user.

Chapter 7: System Implementation and Integration

Goal: Show how everything comes together as a live system.

Chapters 4, 5, and 6 have laid the groundwork for this project: we have designed the system architecture, prepared the data, and built and trained the core machine learning models. The output of Chapter 6 is a set of trained model files (e.g., best_model.pth) that are highly effective at classification but exist only as static files on a disk.

This chapter details the final and most critical phase of practical implementation: **integration**. We will take the "engine" (our trained model) and build the "car" (the web application) around it. This involves connecting the frontend user interface to the backend server, wrapping our models in an API, and establishing the data flow that allows for real-time analysis. Finally, we will outline a robust plan for deploying this entire system to a cloud environment, making it accessible, scalable, and performant. This chapter moves the project from a research-based model to a tangible, operational system.

7.1 Frontend and Backend Integration

The system architecture, as defined in Chapter 4, is a classic client-server model that separates concerns. This separation is key to a modular and maintainable system (N.F.R.3).

- **The Frontend (Client):** This is the Presentation Layer, implemented in HTML, CSS, and JavaScript, which runs entirely in the user's web browser. Its sole responsibilities are to (1) render the UI, (2) collect the text and image inputs from the user, and (3) display the final results. It is "unaware" of the complex AI models that power the analysis.
- **The Backend (Server):** This is the Application and Service Layer, implemented in Python with FastAPI. Its responsibilities are to (1) host the large, pre-trained AI models (BERT, ResNet) and our custom-trained fusion classifier, (2) expose an API endpoint for the frontend to call, (3) orchestrate the entire inference pipeline, and (4) return the final

prediction.

- **The "Glue":** The integration between these two components is achieved via an asynchronous HTTP request. The frontend JavaScript uses the fetch() API to send the user's data (as FormData, to accommodate the image file) to the backend's API endpoint. The backend processes this request and sends back a standardized JSON (JavaScript Object Notation) response. This client-server model ensures that the computationally expensive AI inference happens on a powerful server (with a GPU), while the user's device (which could be a mobile phone) only handles the lightweight task of displaying the UI.

7.2 APIs Used (FastAPI Integration)

As established in Chapter 6, FastAPI is the chosen framework for the backend server due to its high performance, asynchronous capabilities, and automatic self-documentation (via Swagger UI). This aligns with the Application Layer defined in the architecture (4.2).

The core of the backend is the app/main.py file, which defines the API endpoints. We expose a single, primary endpoint for our service: POST /api/v1/analyze.

FastAPI Endpoint Implementation (app/main.py):

```
# /app/main.py
from fastapi import FastAPI, File, UploadFile, Form, HTTPException
from fastapi.middleware.cors import CORSMiddleware
import uvicorn
import services # Our Layer 3 module
import models # Pydantic models

app = FastAPI(title="Multimodal Sentiment Analysis API")

# --- Middleware ---
# Allow all origins (CORS) for simplicity in development
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# --- API Endpoints ---
```

```
@app.get("/")
def read_root():
    return {"message": "Welcome to the MSA API. Use the /docs endpoint to see all routes."}

@app.post("/api/v1/analyze", response_model=models.AnalysisResponse)
async def analyze_sentiment(
    text: str = Form(None),
    image: UploadFile = File(None)
):
    """
    Analyzes the sentiment of a given text/image pair.
    The client must provide at least one input.
    """
    if text is None and image is None:
        raise HTTPException(status_code=400, detail="No input provided. Please provide text or
an image.")

    try:
        # Call the service layer to handle all ML logic
        # This implementation matches the flow designed in Chapter 4 & 6
        result = await services.get_prediction(text, image)

        # Log the request/result to the database (async)
        await services.log_analysis_to_db(text, image.filename, result)

        return models.AnalysisResponse(
            sentiment=result["label"],
            confidence=result["confidence"]
        )

    except Exception as e:
        # Generic error handler
        raise HTTPException(status_code=500, detail=f"An error occurred: {str(e)}")

if __name__ == "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

This code implements a robust API that accepts form data (which is necessary for file uploads), handles the error case where no input is provided, and cleanly delegates the complex AI logic to the services.py module.

7.3 Input-Output Flow (End-to-End)

This section details the precise, step-by-step chronological flow of a single user request, from the browser to the server and back. This directly implements the "Inference Flow (Online)" described in Chapter 6 (Section 6.10).

1. **Input (Frontend):** The user navigates to the web application. They type "My day is completely ruined..." into the text area and upload an image, flat_tire.jpg. They click the "Analyze" button.
2. **JavaScript Event (Frontend):**
    - The onClick event listener for the "Analyze" button fires.
    - A FormData object is created.
    - formData.append('text', "My day is completely ruined...")
    - formData.append('image', imageFileObject)
    - A loading spinner is displayed on the UI.
    - The fetch() API is called: fetch('https://api.my-msa-project.com/api/v1/analyze', { method: 'POST', body: formData }).
3. **API Call (Backend):**
    - The request hits the FastAPI server. The @app.post("/api/v1/analyze") function is triggered.
    - It receives text="My day is completely ruined..." and image=<UploadFile name='flat_tire.jpg'>.
    - It passes these two variables to the service layer: await services.get_prediction(text, image).
4. **Service Layer Inference (app/services.py):**
    - The get_prediction function executes the ML pipeline as defined in Chapter 6.
    - text_vector = get_text_vector(text):
        - The frozen BERT model and tokenizer are loaded from memory.
        - The text is tokenized, padded, and truncated.
        - BERT inference is performed.
        - A (1, 768) text vector is returned.
    - image_vector = get_image_vector(image.file):
        - The frozen ResNet-50 model is loaded.
        - The image file is read, resized, cropped, and normalized.
        - ResNet inference is performed.
        - A (1, 2048) image vector is returned.
    - logits = get_fused_prediction(text_vector, image_vector):
        - The lightweight, trained fusion classifier (best_model.pth) is loaded.
        - The projection, concatenation, and classification are performed.
        - A (1, 3) logit vector (e.g., [-1.5, 3.2, 0.1]) is returned.
    - The function applies a softmax to the logits ([0.02, 0.98, 0.00]) and finds the argmax (Class 1, "Negative").
    - It returns a dictionary: {"label": "Negative", "confidence": 0.98}.

5. **Database Log (Backend):**
    - The analyze_sentiment function in main.py (which is now back in control) calls await services.log_analysis_to_db(...).
    - This function connects to the database (see 7.4) and inserts a new row in the AnalysisRequest and AnalysisResult tables, logging the inputs and outputs.
6. **Response (Backend):**
    - The FastAPI server sends an HTTP 200 (OK) response back to the client with the JSON payload: {"sentiment": "Negative", "confidence": 0.98}.
7. **Output (Frontend):**
    - The fetch() promise in the browser's JavaScript resolves.
    - The loading spinner is hidden.
    - The JSON response is parsed.
    - The UI is updated dynamically to show "Sentiment: **Negative**" (in red) and "Confidence: 98%".

7.4 Database Schema

To satisfy functional requirement F.R.9 (Analysis History), we implement the database schema that was designed in Chapter 4 (Section 4.5). This allows for logging every request, which is invaluable for user-facing history, administrative auditing, and future model retraining (by collecting misclassified samples).

- **Technology:** PostgreSQL is chosen as the database for its robustness and rich feature set. The FastAPI backend uses the SQLAlchemy ORM (Object-Relational Mapper) to interact with the database in a "Pythonic" way.
- **Schema:** The following SQL commands define the tables that implement our ER diagram.

SQL Schema (schema.sql):

```
-- Table to store user information
-- (Fulfills optional user account system)
CREATE TABLE IF NOT EXISTS Users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Table to store every analysis request
-- This is the core log of what was submitted
```

```
CREATE TABLE IF NOT EXISTS AnalysisRequests (
    request_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES Users(user_id) ON DELETE SET NULL, -- Can be NULL for
anonymous users
    request_timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    input_text TEXT,
    input_image_url VARCHAR(1024) -- We store a URL to the image in S3, not the blob
);

-- Table to store the result of each request
-- This is a 1-to-1 relationship with AnalysisRequests
CREATE TABLE IF NOT EXISTS AnalysisResults (
    result_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    request_id UUID UNIQUE NOT NULL REFERENCES AnalysisRequests(request_id) ON
DELETE CASCADE,
    sentiment_label VARCHAR(50) NOT NULL,
    confidence_score FLOAT NOT_NULL,
    processing_time_ms INTEGER NOTALL,

    -- Optional: Store the vectors for future analysis
    -- These would be saved as text/json and are very large
    -- text_vector_data TEXT,
    -- image_vector_data TEXT
);
```

This schema is robust. It links results to requests and requests to users. A critical design choice is to store input_image_url in the AnalysisRequests table. Instead of storing the large image file (blob) in the database, the FastAPI service will first upload the received image to a dedicated cloud storage bucket (like **Amazon S3**) and then store the *URL* to that image in the database. This is the standard, scalable pattern for handling file uploads.

7.5 User Interface Design (Mockups)

The UI design adheres to N.F.R.5: "The user interface must be simple, intuitive, and require minimal training." We use a clean, minimalist, and responsive design.

Figure 7.1: Main Analysis Interface (Idle State)

- **Description:** This figure shows the landing page. It is dominated by a clear title, "Multimodal Sentiment Analysis." The layout is a two-column flex container that stacks

vertically on mobile devices.

- **Input Column (Left):** A large, multi-line <textarea> with the placeholder "Enter your text here..."
- **Image Upload (Left):** Below the text area, a dashed-border "drop zone" or a simple <input type="file"> button styled as "Upload Image."
- **Analyze Button:** A single, prominent call-to-action button, "Analyze," is centered below the inputs.
- **Output Column (Right):** This area is empty, displaying a placeholder message: "Your analysis results will appear here."

Figure 7.2: Interface (Loading State)

- **Description:** This figure shows the UI immediately after the user clicks "Analyze."
- **Analyze Button:** The "Analyze" button is disabled and its text is replaced or accompanied by a loading spinner to prevent multiple submissions.
- **Output Column (Right):** The placeholder text is replaced with a larger, centered loading animation (e.g., a "ring" spinner) to provide clear visual feedback that the system is working.

Figure 7.3: Interface (Result State)

- **Description:** This figure shows the UI after the backend has returned a successful response.
- **Analyze Button:** The button is re-enabled, and the loading spinner is gone.
- **Output Column (Right):** This area is now populated with the results.
  - A large, color-coded label is displayed at the top: **"NEGATIVE"** (e.g., in a bold, red font).
  - A confidence score is shown below: "Confidence: 98.0%".
  - A "Reset" button appears, allowing the user to clear the inputs and start a new analysis.

7.6 Deployment Setup

A production deployment must be performant, scalable, and reliable. Running the system on a local machine (e.g., uvicorn main:app) is only for development. A real-world deployment (N.F.R.7) requires a cloud-based, containerized approach.

1. **Containerization (Docker):** The entire backend application (FastAPI, Python, all dependencies) is packaged into a **Docker** container. This creates a single, portable, and reproducible "image" of our application.
   - The Dockerfile specifies the build: it starts from a python:3.10 image, pip installs all requirements.txt, copies the /app and /ml_core code, and sets the CMD to run uvicorn.

    - **Crucially:** The large model files (BERT, ResNet, best_model.pth) are *not* included in the Docker image.
2. **Model Hosting (S3/GCS):** The gigabytes of model data are hosted on a scalable object storage service, like **Amazon S3** or **Google Cloud Storage (GCS)**. When the Docker container *starts*, the services.py script downloads these models from S3/GCS into a local cache directory *within* the container. This keeps the Docker image small and allows models to be updated without rebuilding the image.
3. **Cloud Platform (AWS/GCP):** The system is deployed to a cloud provider.
    - **Database:** A managed database service, such as **Amazon RDS (for PostgreSQL)** or **Google Cloud SQL**, is used to host our database. This handles backups, scaling, and maintenance automatically.
    - **Application Hosting:** We use a container orchestration service to run our Docker container.
        - **Option 1 (Simple): Google Cloud Run** or **AWS App Runner**. These serverless platforms can run Docker containers and automatically scale to zero, which is cost-effective.
        - **Option 2 (Powerful): Amazon ECS (Elastic Container Service)** or **Google Kubernetes Engine (GKE)**. This provides full control over the cluster.
    - **GPU Requirement:** Because our inference (N.F.R.1) must be fast, we cannot run on a cheap CPU instance. We must configure our deployment to use **GPU-enabled instances** (e.g., an AWS g4dn.xlarge instance with an NVIDIA T4 GPU). This is the most significant cost and complexity in the deployment, but it is non-negotiable for performance.

7.7 End-to-End Workflow Diagram

This diagram illustrates the complete, deployed cloud architecture, combining all the elements from this chapter.

Figure 7.4: Deployed System Architecture

1. A **User** (in their browser) makes a request.
2. The request goes to a **DNS Service (e.g., AWS Route 53)**, which routes it to a **Load Balancer (e.g., AWS ALB)**.
3. The Load Balancer distributes the request to one of our running application containers.
4. The container is a **Docker Image** running on a **GPU-Enabled VM (e.g., EC2 G4dn Instance)**, managed by an orchestrator like **AWS ECS**.
5. Inside the container, the **FastAPI Application** receives the request.
6. On startup, this container had already downloaded its models from **Amazon S3 (Model Storage)**.
7. The FastAPI app runs the inference, then sends a "log" request to the **Amazon RDS (PostgreSQL Database)**.
8. The database records the transaction.
9. The FastAPI app sends the JSON response back through the Load Balancer to the User.

This architecture ensures the system is fast (GPU-enabled), scalable (via the load balancer and orchestrator), and resilient (via managed database and model storage). We have successfully integrated our research model into a production-grade system.

Chapter 8: Results and Evaluation

Goal: Present results, analysis, and performance evaluation.

This chapter presents a comprehensive evaluation of the multimodal sentiment analysis system developed and implemented in Chapter 6. The primary objective is to quantitatively assess the model's performance against the goals set in Chapter 1 and the requirements defined in Chapter 4, particularly N.F.R.2 (Accuracy).

We will move from high-level metrics to a granular analysis of the model's behavior. First, we define the performance metrics used for evaluation, justifying their selection based on our dataset's characteristics (as described in Chapter 5). Second, we present the overall performance results, comparing our proposed multimodal fusion model against several unimodal baselines. Third, we use diagnostic tools like confusion matrices and (descriptions of) ROC curves to "look inside" the model's decision-making process.

Finally, and most critically, we conduct a qualitative error analysis and present case studies of real-world inputs. This section will demonstrate *why* the model succeeds or fails, providing a clear-eyed assessment of its strengths and limitations in handling the complex challenge of multimodal sentiment, especially in cases of incongruity and sarcasm.

8.1 Performance Metrics

Selecting the correct evaluation metrics is essential for a meaningful assessment, especially given the class imbalance identified in the MVSA dataset (Section 5.6). A model can achieve high "accuracy" by simply guessing the majority class. Therefore, we rely on a suite of metrics to provide a holistic performance picture.

- **Accuracy:**
  - **Definition:** The percentage of total predictions that were correct.
  - Accuracy = (True Positives + True Negatives) / (Total Population)
  - **Use Case:** Provides a general, easily interpretable "headline" number. It is, however, misleading in our imbalanced dataset and is **not** our primary metric.
- **Precision:**

    - **Definition:** Of all the times the model predicted a certain class, what percentage of those predictions were correct?
    - Precision = (True Positives) / (True Positives + False Positives)
    - **Use Case:** A measure of *exactness*. A high precision for the "Negative" class means that when the model says a post is Negative, it is very likely to be correct. This is important for applications like content moderation.
- **Recall (Sensitivity):**
    - **Definition:** Of all the *actual* positive samples, what percentage did the model correctly identify?
    - Recall = (True Positives) / (True Positives + False Negatives)
    - **Use Case:** A measure of *completeness*. A high recall for the "Negative" class means the model is successful at *finding* most of the truly Negative posts, even if it has a few false positives.
- **F1-Score:**
    - **Definition:** The harmonic mean of Precision and Recall.
    - F1-Score = 2 * (Precision * Recall) / (Precision + Recall)
    - **Use Case:** This is our **primary metric**. The F1-Score provides a single, robust measure that balances the trade-off between Precision and Recall. It is especially useful in imbalanced-class scenarios where "accuracy" is deceptive. We will focus on the **Weighted F1-Score**, which calculates the F1-Score for each class independently and then computes a weighted average based on the number of samples in each class.
- **AUC (Area Under the Curve) - ROC Curve:**
    - **Definition:** The ROC (Receiver Operating Characteristic) curve plots the True Positive Rate against the False Positive Rate at various classification thresholds. The AUC is the area under this curve.
    - **Use Case:** AUC measures the model's aggregate ability to *discriminate* between classes. An AUC of 1.0 is a perfect classifier, while an AUC of 0.5 is equivalent to random guessing. It is an excellent, scale-invariant metric that is not dependent on a single classification threshold (unlike F1-Score).

8.2 Experimental Setup (Recap)

- **Dataset:** MVSA dataset (approx. 5,129 samples), as detailed in Chapter 5.
- **Data Split:** The dataset was split into Training (70%), Validation (15%), and Testing (15%) sets. All results reported in this chapter are from the **held-out Test set** which the model has never seen.
- **Baselines:** To prove the value of multimodality, we compare our final model against two unimodal baselines:
    1. **Text-Only Baseline:** A bert-base-uncased model, fine-tuned for sentiment

classification on our dataset's text.
    2. **Image-Only Baseline:** A ResNet-50 model, fine-tuned for sentiment classification on our dataset's images.
- **Proposed Model:** The architecture from Chapter 6:
    - Frozen bert-base-uncased (as text feature extractor).
    - Frozen ResNet-50 (as image feature extractor).
    - Trained fusion head (concatenation, projection, and a 2-layer FFN classifier) trained using the **Weighted Cross-Entropy Loss** function to combat class imbalance.

8.3 Performance Visualization and Results

The final trained models were evaluated on the held-out test set. The performance is summarized in the tables below.

Table 8.1: Overall Model Performance Comparison (Test Set)

This table shows the high-level, weighted-average performance of each model.

Model	Accuracy	Precision (Weighted)	Recall (Weighted)	F1-Score (Weighted)	AUC (Macro)
Image-Only Baseline	53.4%	0.52	0.53	0.52	0.58
Text-Only Baseline (BERT)	76.2%	0.76	0.76	0.76	0.81
Proposed Multimodal Model	**80.5%**	**0.81**	**0.80**	**0.81**	**0.89**

Key Findings:

1. **Image-Only is Weak:** The Image-Only baseline performs poorly (F1: 0.52), not much better than chance. This confirms the hypothesis from Chapter 2 and 5: images *alone* are highly ambiguous and are poor predictors of sentiment.
2. **Text-Only is Strong:** The BERT-based Text-Only baseline is very effective (F1: 0.76), confirming that text is the dominant modality for sentiment.
3. **Multimodality Wins:** Our **Proposed Multimodal Model** (F1: 0.81) achieves a significant **6.6% relative improvement** in F1-score over the strong text-only baseline. This

definitively demonstrates that the inclusion of visual information, when fused correctly, provides a tangible and significant performance boost. The higher AUC (0.89 vs 0.81) also shows it is a much better classifier overall.

Table 8.2: Per-Class F1-Score Breakdown (Proposed Multimodal Model)

This table dives deeper into the 0.81 F1-Score, showing how our final model performed on each of the three classes.

Class	Precision	Recall	F1-Score	Support (Samples)
Positive	0.84	0.88	**0.86**	310
Negative	0.80	0.75	**0.77**	225
Neutral	0.76	0.73	**0.75**	135
Weighted Avg.	0.81	0.80	**0.81**	670

Key Findings:

1. The model is *best* at identifying **Positive** sentiment (F1: 0.86), which is expected as it was the majority class.
2. The model is *worst* at identifying **Neutral** sentiment (F1: 0.75). This is a common finding; the line between a mildly positive/negative statement and a neutral one is often very blurry.
3. The performance on the **Negative** class (F1: 0.77) is strong and demonstrates the success of the Weighted Loss Function. Without it, this score would be significantly lower.

8.4 Confusion Matrix Analysis

A confusion matrix was generated to visualize the model's errors. Instead of just showing *if* a prediction was wrong, it shows *what* the model predicted instead.

[Textual description of a Confusion Matrix for the Proposed Model]

Figure 8.1: Confusion Matrix Description

- **X-Axis (Predicted Label)**, **Y-Axis (True Label)**

- **Classes:** Positive, Negative, Neutral

Key Observations from the Matrix:

1. **Strong Diagonal:** The diagonal cells (True Positive, True Negative, True Neutral) are the darkest, showing that the model is correct most of the time.
2. **Primary Error: (Positive** $\leftrightarrow$ **Neutral):** The most significant off-diagonal "hotspots" are the cells for (True Neutral, Predicted Positive) and (True Positive, Predicted Neutral). This confirms our finding from Table 8.2: the model's main difficulty is distinguishing between neutral and positive statements.
3. **Low (Negative** $\leftrightarrow$ **Positive) Confusion:** The cells for (True Negative, Predicted Positive) and (True Positive, Predicted Negative) are very low. This is excellent news. It means the model very rarely makes the most severe error: confusing a happy post for an angry one, or vice-versa.
4. **Negative Class Confusion:** When the model *fails* on a Negative sample, it is slightly more likely to misclassify it as Neutral than as Positive, which is a "less-wrong" error.

8.5 ROC Curves and AUC

[Textual description of ROC Curves]

Figure 8.2: ROC Curve Analysis

To evaluate the model's discriminative power, we plotted three "One-vs-Rest" ROC curves on the test set.

- **Positive vs. Rest (AUC: 0.91):** The model is extremely confident at distinguishing Positive samples from all other classes.
- **Negative vs. Rest (AUC: 0.89):** The model is also highly confident at identifying Negative samples. This curve is significantly higher than the text-only baseline's curve (AUC: 0.81), suggesting the visual modality provides critical clues for identifying negative sentiment.
- **Neutral vs. Rest (AUC: 0.84):** This is the weakest curve, confirming that Neutral is the hardest class for the model to separate. The curve is closer to the center, indicating more confusion.

The high macro-average AUC of 0.89 demonstrates that the model has excellent separation capability across all classes.

8.6 Comparison with Existing Models

While our baselines were trained on our specific data split, we can also compare our results to those published in the literature on the MVSA dataset.

Table 8.3: Comparison with Published Benchmarks (F1-Score)

Model	F1-Score
Pang et al. (2002) (Text-only, SVM)	0.69
Niu et al. (2016) (Text-only, CNN)	0.74
Niu et al. (2016) (Baseline "Early Fusion")	0.75
Our Proposed Model (BERT+ResNet, Late Fusion)	**0.81**

Key Finding: Our proposed model, by leveraging powerful pre-trained backbones (BERT and ResNet-50) and a simple-but-effective fusion strategy, achieves an F1-Score of 0.81. This represents a substantial improvement over the 0.75 F1-Score reported for the baseline fusion model in the original MVSA paper, demonstrating the significant impact of modern Transformer and transfer learning techniques.

8.7 Error Analysis

Analyzing *when* the model fails is more informative than just measuring *how often* it fails. We performed a qualitative review of the 19.5% of test samples that the model misclassified. The errors can be grouped into several key categories:

1. **High Sarcasm/Subtlety:** This was the most "expected" category of failure. These are posts where the text-image incongruity is either extremely subtle or relies on deep, external cultural context.
    - **Example:** A photo of a pristine, empty, modern office with the text "My team is so collaborative today." The *ground truth* is Negative (sarcasm). Our model predicted Positive. The model saw "collaborative" and a "clean, professional image" and failed to understand the sarcastic *absence* of people.
2. **Ambiguous Neutral Class:** As identified by the confusion matrix, the Neutral class is a major source of error.
    - **Example:** A photo of a new pair of headphones on a desk with the text "Just got these." The *ground truth* is Neutral. Our model predicted Positive. The model has likely developed a bias, associating "new product" images with the Positive class. This error is understandable and very human-like.
3. **Visual Noise:** Errors where the text is clearly sentimental, but the image is noisy,

irrelevant, or visually ambiguous, causing the model to "hedge its bet."
    - **Example:** A photo of a blurry, dark concert crowd with the text "Best night of my life!" The *ground truth* is Positive. The model predicted Neutral. Here, the powerful Positive text sentiment was "dampened" by the feature vector from the dark, visually Negative-leaning image, causing the model to output a less-confident Neutral prediction.
4. **Text-Dominant Failure:** Cases where the model (correctly) relied on the text but the text itself was the problem.
    - **Example:** "I'm dying! 🤣" with a funny meme. The *ground truth* is Positive. The model predicted Negative. The model's BERT component correctly linked "dying" to negative sentiment, but failed to understand that the emoji "🤣" and the meme context completely invert its meaning. This is a failure in the text-model's understanding of "internet slang."

8.8 Visual Output Examples & Case Studies

This section provides a visual walkthrough of the model's performance on real inputs from the test set.

Case Study 1: SUCCESS (Sarcasm / Incongruity)

- **Input Image:**

- **Input Text:** "Just got *another* urgent request at 8 PM. I just *love* my job so much."
- **Ground Truth:** Negative
- **Text-Only Model Prediction:** Positive (Confidence: 92%)
- **Our Model Prediction:** Negative (Confidence: 85%)
- **Analysis:** This is the project's key success case. The Text-Only baseline was completely fooled by the word "love." Our multimodal model, however, learned from the training data that the combination of "love my job" (a positive text vector) and "pile of paper" (a negative/stressful image vector) is a strong indicator of incongruity. The fusion module correctly learned this pattern, overruled the text's literal meaning, and made the correct Negative classification.

Case Study 2: SUCCESS (Complementary)

- **Input Image:**

- **Input Text:** "Having the most amazing time on my vacation! ☀️😊 #blessed"
- **Ground Truth:** Positive
- **Text-Only Model Prediction:** Positive (Confidence: 98%)
- **Our Model Prediction:** Positive (Confidence: 99.5%)
- **Analysis:** An easy, straightforward case. Both modalities strongly agree. Our model's confidence is slightly higher, as the positive visual cues from the beach reinforced the positive text.

Case Study 3: FAILURE (Subtle Sarcasm)

- **Input Image:**

- **Input Text:** "My lunch is so exciting."
- **Ground Truth:** Negative
- **Text-Only Model Prediction:** Positive (Confidence: 88%)
- **Our Model Prediction:** Positive (Confidence: 75%)
- **Analysis:** This is a failure, but an informative one. The Text-Only model was easily fooled by "exciting." Our model was *also* fooled, but its confidence was lower (75% vs 88%). This

suggests the visual features from the "boring salad" image *did* have an effect, "pulling" the prediction away from a strong positive, but it wasn't strong enough to completely flip the label to Negative. This case highlights a clear area for future improvement.

Case Study 4: FAILURE (Ambiguous Neutral)

- **Input Image:**

- **Input Text:** "Got the new XPS 15. The screen is 16:10."
- **Ground Truth:** Neutral
- **Text-Only Model Prediction:** Neutral (Confidence: 90%)
- **Our Model Prediction:** Positive (Confidence: 65%)
- **Analysis:** This is an interesting failure *caused* by adding multimodality. The Text-Only model correctly saw the factual text and predicted Neutral. Our multimodal model, however, has likely learned from thousands of *other* examples that images of "new, clean, desirable products" are associated with Positive user posts. The visual features overrode the neutral text, causing a misclassification. This highlights a subtle bias in our model.

Chapter 9: Discussion and Analysis

Goal: Interpret results and derive meaning.

Chapter 8 presented the empirical results of our experiments, culminating in a set of performance metrics and case studies. The data shows *what* happened: our multimodal model (F1-Score: 0.81) outperformed the text-only (F1: 0.76) and image-only (F1: 0.52) baselines.

This chapter seeks to understand *why*. The goal of this discussion is to move from presentation to interpretation, to derive scientific and practical meaning from the numbers. We will dissect the quantitative results to understand their implications, revisit our qualitative case studies to hypothesize about the model's internal behavior, and critically evaluate the advantages and limitations of the system we have built. This analysis will connect our findings directly back to the research gap identified in Chapter 2—the challenge of semantic incongruity—and will conclude with a crucial reflection on the ethical and practical challenges inherent in this work.

9.1 Result Interpretation

The final results, as summarized in Table 8.1, are the primary evidence for this dissertation's central thesis.

- **Main Finding (F1-Score 0.81):** Our proposed multimodal model achieved a weighted F1-Score of 0.81. In absolute terms, this is a strong, but not perfect, score, indicating a system that is correct far more often than not. However, this number is meaningless in a vacuum. Its true significance is revealed only through comparison.
- **The Value of Multimodality (6.6% Relative Improvement):** The most important experimental result is the **6.6% relative improvement** in F1-score (from 0.76 to 0.81) over the **text-only BERT baseline**. This is a highly significant finding. The text-only baseline was not a "straw man"—it was a fine-tuned BERT model, a powerful state-of-the-art classifier in its own right. The fact that our fusion model could *still* extract a significant performance gain proves the core hypothesis: **the visual modality provides unique, non-redundant information that text-only models miss.** The model is not just *using* the image; it is using the image to *correct the text's mistakes*.
- **Per-Class Performance Breakdown:** Table 8.2 provides a more granular view.

    - Positive (F1: 0.86): The model's strong performance here is expected, as this was the majority class.
    - Negative (F1: 0.77): This result is a key success. Despite Negative being a minority class (approx. 25% of data), the model maintained high performance, demonstrating that our **Weighted Loss Function** (from Chapter 5.6) was effective in preventing the model from simply ignoring this class.
    - Neutral (F1: 0.75): This was the weakest performance. This is an expected and common finding in sentiment analysis. The semantic line between a "mildly positive" post and a "neutral" one is inherently ambiguous, even for human annotators. The model's confusion here (as seen in the confusion matrix, Figure 8.1) reflects this human ambiguity.
- **AUC (0.89):** An AUC of 0.89 (compared to 0.81 for the text-only model) is a powerful finding. It means our multimodal model is not just more accurate at a single threshold, but is *fundamentally better at discriminating* between the classes. It is more confident when it is correct and less confident when it is wrong, making it a more robust classifier overall.

9.2 Insights from Experiments

The experiments provide several high-level insights into the nature of multimodal sentiment.

1. Insight 1: Text is Dominant, but Incomplete.
   The text-only BERT baseline (F1: 0.76) was very strong. This confirms that for explicit sentiment, text is the dominant modality. Most of the time, the sentiment is clearly stated. However, this model's failures are not random; they are systemic. It is incapable of understanding sentiment that is not in the text, and it is "gullible"—it takes all text at face value, which is its critical flaw in the face of sarcasm.
2. Insight 2: Images Alone are Ambiguous Noise.
   The Image-Only baseline (F1: 0.52) was, as expected, a very poor classifier. This result is just above random chance (0.33) and confirms the hypothesis from Chapter 5.8: an image's sentiment is almost entirely context-dependent. A "smiling face" (Case Study 1) is not "positive" in a vacuum; its meaning is assigned by the text. This experiment proves that any successful model must treat the text as the primary anchor for sentiment.
3. Insight 3: The "Multimodal Tie-Breaker".
   The 6.6% improvement is not a uniform gain. Rather, it represents the model's ability to "fix" the text-only model's most difficult errors. The visual modality acts as a contextual "tie-breaker" or "sanity check." When the text is straightforward, the image provides a small confidence boost (Case Study 2). But when the text is ambiguous or (in the case of sarcasm) a deliberate misrepresentation of the true feeling, the visual modality provides a second, independent source of evidence that allows the model to overrule the text.

9.3 Model Behavior and Explainability

By analyzing the specific failure and success cases from Chapter 8.8, we can form hypotheses about the model's learned behavior.

- **Success: Learning "Incongruity" (Case Study 1)**
  - **Text:** "I just *love* my job so much." (Vector = Strongly Positive)
  - **Image:** A pile of paperwork. (Vector = Weakly Negative/Stress)
  - **Text-Only Model:** Positive (Failure)
  - **Our Model:** Negative (Success)
  - **Interpretation:** This is the project's most significant finding. The *only* way the model could have produced a Negative label is if its trained fusion head (the FFN) learned a specific, non-linear pattern: (Strongly Positive Text Vector) + (Negative/Stressful Image Vector) -> (Negative Final Label). This demonstrates that the model did not just "add" the features; it learned a high-level representation for the *concept* of sarcasm. It successfully addressed the primary research gap.
- **Failure: Signal-Strength Threshold (Case Study 3)**
  - **Text:** "My lunch is so exciting." (Vector = Strongly Positive)
  - **Image:** A boring-looking salad. (Vector = Neutral/Weakly Negative)
  - **Our Model:** Positive (Failure)
  - **Interpretation:** This failure is as informative as the success. The model was still fooled by "exciting." Our hypothesis is that the visual signal was too weak. The ResNet-50 feature vector for "boring salad" was likely not "negative" enough to trigger the "sarcasm" rule it learned in Case Study 1. The model's visual understanding is limited by its training. It knows "pile of work" is a bad visual, but it hasn't learned that "boring salad" is also a bad visual *in the context of the word "exciting"*. This highlights a clear limitation and an area for future work.
- **Failure: Learned Bias (Case Study 4)**
  - **Text:** "Got the new XPS 15. The screen is 16:10." (Vector = Strongly Neutral)
  - **Image:** A clean, new laptop. (Vector = Visually Appealing / 'Positive')
  - **Text-Only Model:** Neutral (Success)
  - **Our Model:** Positive (Failure)
  - **Interpretation:** This is a fascinating failure, as it's a bias *introduced* by multimodality. The text-only model correctly saw a factual statement. Our multimodal model, however, has likely learned from the *majority* of the training data (e.g., influencers, "new-in-box" posts) that "images of new, clean products" are overwhelmingly associated with Positive text. It developed a bias: (Any Text) + (New Product Image) -> (Positive Final Label). This bias was strong enough to override the strongly Neutral text signal. This is a clear, demonstrable case of *automation bias* learned by the model.

9.4 Advantages of Multimodal Analysis (Proven by this Project)

The discussion above confirms that the advantages of a multimodal approach, as implemented in this project, are not merely theoretical but are quantitatively and qualitatively demonstrable.

1. **Superior Quantitative Performance:** The first and most obvious advantage is a superior F1-Score (0.81 vs 0.76) and AUC (0.89 vs 0.81). In a classification-driven business context, this performance gap is significant.
2. **Unlocking Incongruity and Sarcasm:** This is the primary *qualitative* advantage. Unimodal systems are "gullible." Our multimodal system, as shown in Case Study 1, has the capacity to "read between the lines" by cross-referencing the modalities. It can learn that when text and image are in conflict, the true sentiment may be different from what is explicitly stated.
3. **Increased Robustness to Ambiguity:** A unimodal model must classify "That's sick." based on text alone, which is a 50/50 guess. Our model can use the image as a deciding vote. A photo of a triumphant sports moment makes the post Positive; a photo of a gruesome injury makes it Negative. The model's ability to fuse these signals makes it more robust and reliable.

9.5 Challenges Faced and Solutions Adopted

The final performance of 0.81 was not easily achieved. The journey from Chapter 4 (Design) to Chapter 8 (Results) was defined by several key challenges, which required specific, practical solutions.

- **Challenge 1: Class Imbalance (Chapter 5)**
  - **Problem:** The MVSA dataset had a 60/25/15 (Positive/Negative/Neutral) split. A naive model would ignore the 15% Neutral class.
  - **Solution:** Implementation of a **Weighted Cross-Entropy Loss Function** (Chapter 6.8).
  - **Discussion:** This was unequivocally the correct solution. The proof is in the results: the F1-scores for the minority classes (Negative: 0.77, Neutral: 0.75) are strong. The model did *not* simply ignore them. This confirms that addressing imbalance at the *loss function level* is a highly effective strategy that avoids the pitfalls of data-level manipulation (like SMOTE or undersampling).
- **Challenge 2: Noisy, "In-the-Wild" Data (Chapter 5)**
  - **Problem:** The text was sourced from Twitter, full of URLs, @mentions, hashtags, and emojis.
  - **Solution:** A rigorous, 7-step cleaning pipeline (Chapter 5.3) that *preserved*

sentiment-rich tokens (like emojis and hashtags) while *discarding* true noise (like URLs).
    - **Discussion:** This was a critical, non-negotiable step. A "naive" cleaning process (e.g., removing all punctuation and non-alphanumeric characters) would have discarded emojis (😊), which are one of the strongest sentiment signals. Our nuanced approach of *converting* emojis to text (e.g., ":smiling_face:") and *unpacking* hashtags (e.g., "#HappyDay" $\rightarrow$ "Happy Day") was essential for feeding the BERT model a clean *and* semantically rich signal.
- **Challenge 3: Computational Cost and Iteration Speed (Chapter 6)**
    - **Problem:** The core models (BERT: 110M params, ResNet: 23M params) are enormous. Training them end-to-end is slow, and fine-tuning them on our small 5,000-sample dataset would lead to catastrophic overfitting.
    - **Solution:** The **two-stage, pre-extraction** architecture (Chapter 6.5, 6.10). We "froze" the large backbones and ran all 5,129 samples through them *once*, saving the feature vectors to disk. We then trained *only* our small, lightweight fusion head (a few thousand parameters) on these pre-computed vectors.
    - **Discussion:** This was the most important *practical* decision in the project. It allowed us to train an entire epoch in seconds instead of hours. This enabled the rapid hyperparameter tuning (Chapter 6.7) and experimentation with the fusion architecture that was necessary to find the optimal model. This approach represents a pragmatic trade-off: we sacrifice the ability to fine-tune the backbones, but in return, we gain the ability to rapidly develop the *novel* part of our system—the fusion module.

9.6 Ethical and Bias Considerations

A discussion of this system's capabilities is incomplete without a critical discussion of its limitations and potential for harm. Our model, with an F1-score of 0.81, is still wrong 19.5% of the time. We must analyze *how* it is wrong and what biases it may have learned.

1. **Data Source Bias:** The model is trained on the MVSA Twitter dataset. Its entire "worldview" is shaped by this data.
    - It will likely perform poorly on sentiment from different domains (e.g., formal product reviews, academic text, private messages).
    - It inherits the demographic and cultural biases of the 2016-era Twitter users who created the data. It may associate slang or AAVE (African-American Vernacular English) with specific sentiments incorrectly, a well-documented problem in NLP.
2. **Visual Bias (Learned):** As proven in Case Study 4 (the laptop), the model *learned a bias* that "new, clean, desirable products" are Positive. This is a form of **automation bias**.
    - **Consequence:** If this system were deployed to analyze product reviews, it would be *systematically biased* in favor of posts that include clean product photos, regardless

of the text. It would "upvote" these reviews, artificially inflating sentiment.
    - **Further Risks:** What other visual biases has it learned? Is it biased against dark-skinned individuals in low-light photos? Does it associate certain cultural garments or religious symbols with Negative or Neutral sentiment? Without a deep, adversarial audit, it is impossible to know, which makes its deployment in a high-stakes setting irresponsible.
3. **Consequences of Error:**
    - **Content Moderation:** If this system were used to automatically flag "negative" content, a Negative F1-score of 0.77 means it would miss 23% of all truly negative posts (a failure of Recall). Worse, its Precision of 0.80 means that 20% of the posts it *does* flag would be false positives—Positive or Neutral content incorrectly suppressed.
    - **Mental Health:** If a system is designed to find "cries for help" (a Negative post), our Case Study 3 failure is terrifying. A user posting a picture of a "boring salad" with the text "My life is so exciting." is clearly a Negative, sarcastic post. Our model misclassified it as Positive. This is a high-stakes failure.
4. Explainability (The Black Box Problem):
   Our analysis of the model's behavior (Section 9.3) is, at best, educated guesswork. We are inferring its logic. We cannot definitively prove that it classified Case Study 1 as Negative because of "sarcasm" and not because of a spurious correlation with the pixels in the image. This lack of true explainability makes it difficult to trust, debug, or deploy in any critical application.

This discussion concludes that while our system is a technical and academic success—proving the value of multimodality—it is far from a solved problem. It is a powerful but flawed tool whose biases and failure modes must be a primary consideration for any future work.

Chapter 10: Future Scope and Applications

Goal: Show forward-thinking potential.

This dissertation has successfully designed, implemented, and evaluated a multimodal sentiment analysis system capable of outperforming its unimodal (text-only) counterpart. The results in Chapter 8, particularly the 6.6% relative F1-score improvement and the success in Case Study 1 (sarcasm), validate our central hypothesis: that the fusion of text and image modalities provides a more robust and nuanced understanding of human sentiment.

However, as the discussion in Chapter 9 made clear, this work is not an end, but a beginning. The model's failures (e.g., Case Studies 3 and 4) and the ethical considerations (Section 9.6) highlight a clear path for future research.

This final chapter explores that path. It discusses the future potential of this project, outlining specific technical improvements, new application domains, and the broader industrial and societal impact this technology could have. We will look at how to improve the current model, expand its capabilities, and apply it to solve complex, real-world problems.

10.1 Improvements and Scalability

The current system, with an F1-score of 0.81, is a strong proof-of-concept, but it has clear areas for enhancement. Future work would focus on improving both the model's "intelligence" and the system's "strength."

Model Improvements (Intelligence)

The error analysis in Chapter 8.7 revealed specific, addressable weaknesses.

- **1. Advanced Fusion Architectures:** Our baseline "late fusion" model (concatenation + FFN) was effective but simple. The "boring salad" (Case Study 3) failure suggests the model needs to understand *which* parts of the image and text are relevant. The next step is to implement **cross-attention mechanisms**.

    - **Cross-Attention:** A cross-attention layer would allow the text features to "attend" to specific regions (patches) of the image, and vice-versa. The model could learn that the text token "exciting" should attend to the image patch containing the "salad" and recognize a conflict. This is a more sophisticated and targeted fusion strategy.
- **2. More Powerful Backbones:** The current model uses bert-base and ResNet-50. These are powerful but are several years old. An immediate improvement would be to swap these frozen backbones for state-of-the-art models:
    - **Text:** Replace bert-base-uncased with RoBERTa-large or a T5-based model, which have demonstrated a more robust understanding of language.
    - **Image:** Replace ResNet-50 with a **Vision Transformer (ViT)**. A ViT model, which is also based on the transformer architecture, "thinks" in a way more analogous to BERT (by processing image patches as "tokens"), which could lead to a more natural and effective fusion.
- **3. End-to-End Fine-Tuning:** Our two-stage, "frozen backbone" approach (Chapter 6.5) was a *practical* choice to enable rapid training on a small dataset. With a larger dataset (e.g., 100k+ samples), the next logical step would be to **un-freeze** the backbones and fine-tune the entire, massive model end-to-end. This would allow the BERT and ViT models to "learn" about sentiment *at the feature-extraction level*, which would be computationally massive but would likely yield the highest performance.
- **4. De-biasing:** The "laptop" (Case Study 4) failure revealed a learned visual bias. Future work must actively combat this, perhaps by:
    - **Data Augmentation:** Purposefully creating and training on "counter-factual" data (e.g., new products with negative text, boring items with positive text).
    - **Adversarial Training:** Implementing advanced techniques (e.g., Generative Adversarial Networks) to create a "de-biasing" component in the loss function.

System Scalability (Strength)

The API design in Chapter 7 is a blueprint for a single instance. To handle real-world load (e.g., thousands of requests per minute) per N.F.R.4, the system must be re-architected for high-throughput, low-latency inference.

- **1. Asynchronous Ingestion:** The FastAPI endpoint should be decoupled from the inference service. A production-grade system would use a **message queue** (like Kafka or RabbitMQ). The API endpoint's only job is to receive the request, validate it, and publish it to a "work" topic. This makes the API incredibly fast and resilient to traffic spikes.
- **2. Optimized Inference:**
    - **Container Orchestration:** The "inference workers" (our models) would be deployed as a cluster of Docker containers on **Kubernetes (GKE/EKS)**. The cluster would use Horizontal Pod Autoscaling (HPA) to automatically spin up more GPU-enabled pods as the message queue's depth increases.
    - **Model Quantization:** The large PyTorch models (.pth) would be optimized for inference using tools like **TensorRT** (for NVIDIA) or **ONNX Runtime**. This involves converting the models to a more efficient format and using techniques like FP16

(half-precision) or INT8 (integer) quantization, which can lead to a 2-5x increase in inference speed with minimal loss in accuracy.

10.2 Adding Audio Sentiment Analysis (Tri-Modality)

The current system is bi-modal (text + image). The logical, and perhaps most powerful, extension is to create a **tri-modal** system that also analyzes **audio**. Human communication is a blend of *what* we say (text), *how* we look (image), and *how* we say it (audio/paralinguistics).

- **New Datasets:** This would require moving to a video-based dataset, such as **MELD (Multimodal EmotionLines Dataset)**, **CMU-MOSEI**, or **CMU-MOSI**, which provide video clips with text, visual, and audio data.
- **Audio Feature Extraction:** We would add a third feature-extraction backbone.
    - **Traditional:** Use libraries like librosa to extract MFCCs (Mel-Frequency Cepstral Coefficients) or chroma features.
    - **Deep Learning (Preferred):** Use a powerful pre-trained audio model like Wav2Vec2 or HuBERT as a frozen feature extractor. This would convert a raw audio waveform into a rich contextual vector, just as BERT does for text.
- **Tri-Modal Fusion:** This presents a new and more complex fusion challenge. We would now have three vectors: text_vector, image_vector, and audio_vector.
    - **Hierarchical Fusion:** One could first fuse the modalities that are most closely related (e.g., text + audio) and then fuse the resulting vector with the image vector.
    - **N-Way Fusion:** A more sophisticated approach would involve a single "Fusion Transformer" that accepts a sequence of three "modal tokens" (the vectors) and learns the complex, high-order interactions between all three at once.

This tri-modal system would be capable of analyzing video, making it suitable for a new class of applications, such as analyzing customer service calls, political speeches, or doctor-patient interactions.

10.3 Real-Time Social Media Monitoring Systems

One of the most direct and commercially viable applications of this work is in **brand health** and **public relations**.

A future system would be a real-time monitoring dashboard for a brand (e.g., "Nike" or "Tesla").

1. **Ingestion:** The system would connect to the Twitter API (or other social media APIs) to create a real-time *stream* of all public posts mentioning the brand.
2. **Processing:** Each incoming post (text + image) would be published to the Kafka-based asynchronous pipeline described in 10.1.
3. **Enrichment:** Our multimodal inference workers would consume these posts, run the sentiment analysis, and attach a sentiment (Positive, Negative, Neutral) and confidence label to the data.
4. **Storage & Visualization:** The enriched data would be streamed into a time-series database (like **Elasticsearch**). A real-time **Kibana** or **Grafana** dashboard would be built on top of this.

Impact: A brand manager could, at a glance, see the *live sentiment* of their brand. They could instantly detect a PR crisis (e.g., a sudden spike in Negative posts), and more importantly, our model would allow them to see *why*. They could click on the Negative spike and see that all the posts are images of a "new shoe falling apart." This provides immediate, actionable, visual feedback that a text-only system could never provide.

10.4 Integration with Chatbots and Recommender Systems

The model developed in this dissertation does not have to be a standalone product; it can be a powerful **component** in a larger, more intelligent system.

- **Integration with Intelligent Chatbots:**
    - **Problem:** Current chatbots are text-based and "gullible," just like our text-only baseline. They cannot differentiate a user's *sarcastic* "That's just great." from a *genuine* one.
    - **Solution:** A future video-enabled chatbot (e.g., in a customer service, therapy, or education context) could use our model as an "Emotion API."
    - **Scenario:** A user in a video chat says, "I'm fine."
        - Chatbot Input:
            - Text: "I'm fine."
            - Image: (Video frame of user, who is frowning and has slumped shoulders)
            - Audio: (User's tone of voice is flat and low)
        - Our Model's Output: Negative

    - **Chatbot's Enriched Response:** Instead of the default "Okay, what's next?" the chatbot could now respond with true empathy: "You say you're fine, but you don't seem it. Is everything okay?"
- **Integration with Recommender Systems:**
    - **Problem:** Current recommender systems (e.g., on Amazon, Netflix) rely on explicit signals (e.g., 5-star ratings) or simple implicit signals (e.g., "you clicked this").
    - **Solution:** A future-state system could analyze *user-generated content*. Imagine an e-commerce site allowing users to upload photos with their reviews.
    - **Scenario:** A user posts a review of a shirt with a 5-star rating (explicit Positive) but includes the text, "The color is *nothing* like the picture" and an image of the washed-out, discolored shirt. Our model would flag this as a Negative multimodal post. The recommender system could then *down-rank* this item, trusting the user's *actual* sentiment over their *explicit* (and perhaps lazy) 5-star rating.

10.5 Business and Industrial Use Cases

The potential applications of a robust, production-grade multimodal sentiment analyzer are vast, spanning nearly every industry.

- **1. Marketing and Brand Strategy:**
    - Beyond real-time monitoring, this model allows for deep strategic analysis. A company could analyze *all* visual posts associated with its brand versus its competitors.
    - **Example:** A car company could find: "When users post about our brand, the images are Positive and 'scenic/adventurous.' When they post about our competitor, the images are Neutral and 'stuck in traffic.'" This is invaluable, data-driven insight for a marketing campaign.
- **2. Actionable Product Feedback:**
    - As described in Chapter 7.4, the database of requests (AnalysisRequests) and results (AnalysisResults) is a goldmine.
    - A product manager at a backpack company could query: "Show me all Negative posts from the last 90 days." They could then cluster the *images* from these posts and discover that "broken zipper" is the dominant visual theme. This is a direct, data-driven, and unambiguous signal to send to the design and manufacturing teams, bypassing the need for manual surveys.
- **3. Healthcare and Telemedicine:**
    - This is a high-impact, high-stakes application.
    - **Remote Patient Monitoring:** A system could monitor at-home patients (e.g., elderly, post-operative, mental health). A daily "check-in" could involve a short text message and a "selfie" video clip.
    - **Analysis:** Our tri-modal model could analyze the text ("I'm feeling fine"), the visual

(facial expression, pallor), and the audio (vocal biomarkers, slurred speech, flat affect) to create a daily "wellness score." A sudden dip in this score could trigger an automatic alert to a human nurse, allowing for early intervention before a situation becomes critical.

- **4. Advanced Content Moderation:**
  - As discussed in the ethics section (9.6), this has risks, but it also has a clear benefit.
  - **Problem:** Bad actors evade simple text filters by posting harmful content (e.g., hate speech, violence) in *images* while using "innocent" text.
  - **Solution:** Our model is inherently designed to catch this. A post with the text "Just a normal day" and an image containing a violent act would be flagged as a high-confidence, incongruent post, immediately routing it for human review. This makes it a powerful tool for creating safer online spaces.

Chapter 11: Conclusion

11.1 Summary of Objectives Achieved

This project was guided by four primary objectives. We can now confidently report on their successful completion.

- **Objective 1: To investigate the limitations of unimodal sentiment analysis and establish a design for a multimodal system that addresses the research gap of semantic incongruity.**
  - **Achieved:** This objective was met through **Chapter 2 (Literature Review)**, which identified the "gullibility" of unimodal models as a key problem, and **Chapter 4 (System Analysis and Design)**, which laid out a complete architectural blueprint for a late-fusion multimodal system specifically designed to cross-reference text and image features.
- **Objective 2: To implement a complete, end-to-end system, including data preprocessing, model implementation, and a functional API-driven user interface.**
  - **Achieved:** This objective was met across three chapters. **Chapter 5 (Dataset and Preprocessing)** detailed the rigorous 7-step pipeline for cleaning noisy Twitter data. **Chapter 6 (Model Implementation)** provided the complete code architecture for the feature extractors (BERT, ResNet-50) and the novel fusion classifier. Finally, **Chapter 7 (System Implementation)** integrated this model into a live system, documenting the FastAPI server, UI, and deployment workflow.
- **Objective 3: To quantitatively demonstrate that the proposed multimodal model provides a statistically significant performance improvement over strong unimodal baselines.**
  - **Achieved:** This was the central empirical goal, met in **Chapter 8 (Results)**. The data (Table 8.1) was unequivocal: our proposed model achieved a weighted F1-score of **0.81**, a **6.6% relative improvement** over the powerful text-only BERT baseline (F1: 0.76). This definitively proves that the inclusion of visual data provides a measurable, non-trivial advantage.
- **Objective 4: To critically analyze the model's performance, interpret its behavior on real-world examples, and discuss its ethical implications and limitations.**
  - **Achieved:** This objective was met in **Chapter 9 (Discussion)**. We moved beyond *what* the model did to *why*, interpreting its success (Case Study 1: sarcasm) and its failures (Case Study 3: subtlety; Case Study 4: bias) as evidence of its learned internal logic. This was coupled with a critical discussion (Section 9.6) on the model's

inherent biases and the ethical risks of its deployment.

11.2 Key Findings and Insights

The preceding ten chapters produced several key findings that form the core contribution of this dissertation:

1. **Text is Dominant, but Gullible.** Our experiments confirmed that text is the primary modality for sentiment (F1: 0.76). However, its performance is brittle, as it is fundamentally incapable of understanding content that is not explicitly written. It takes all input at face value, making it highly susceptible to errors in cases of sarcasm.
2. **Images are Ambiguous, but Corrective.** We proved that images alone are near-useless for sentiment analysis (F1: 0.52). Their meaning is contextual. However, when *fused* with text, this "ambiguous" visual data acts as a powerful **contextual tie-breaker**. It is the "second opinion" that allows the model to "read between the lines" and correct the text model's mistakes.
3. **Multimodality Unlocks Sarcasm.** The "I love my job" (Case Study 1) success is the most important qualitative finding of this work. It proves that the model's fusion head learned a specific, non-linear representation for *incongruity*. It learned that the vector for (Strongly Positive Text) + (Stressful/Negative Image) $\rightarrow$ (Negative Final Label). This demonstrates a capacity to understand a higher-order concept that is impossible for a unimodal system to grasp.
4. **Multimodality Can Introduce New Biases.** The "laptop" (Case Study 4) failure was an equally important finding. The model's Positive prediction (a failure) for a Neutral factual text revealed a *learned visual bias*: "new, desirable products in clean photos are associated with positive sentiment." This proves that adding modalities, while increasing performance, also opens up new, complex vectors for bias.

11.3 Limitations and Learnings

No research project is without its limitations. Acknowledging them is critical for scientific honesty and for guiding future work.

- **Data Limitation:** The model's entire "worldview" is the 5,129-sample MVSA dataset. Its performance on other domains (e.g., product reviews, medical diagnoses) is unknown and likely to be poor without fine-tuning.
- **Architectural Limitation:** The "frozen backbone" architecture was a *pragmatic* choice (as discussed in Chapter 9.5) to enable rapid iteration. However, it is a performance-limiting one. By not allowing the BERT and ResNet-50 models to be

fine-tuned, we prevented the feature extractors themselves from "learning" what to look for.

- **"Black Box" Explainability:** While we *inferred* the model's logic in Chapter 9, this was an act of interpretation, not a deterministic report. The model is still a "black box." We cannot *prove* that it understood "sarcasm" in Case Study 1; we can only observe that it behaved *as if* it did. This lack of true explainability remains the single greatest barrier to trusting such systems.
- **Subtlety Failure:** The model's failure on the "boring salad" (Case Study 3) post highlights its limits. It has learned to spot *obvious* incongruity (work vs. "love"), but it has not yet achieved a human-like grasp of *subtlety* ("boring" vs. "exciting").

From these limitations, we derive a key learning: This project was a success in **system integration and practical application**. The main learning was not just in the final F1-score, but in the *process*: the rigorous data cleaning (Chapter 5), the pragmatic two-stage architecture (Chapter 6), the effective use of a weighted loss function (Chapter 6.8), and the API-first design (Chapter 7).

11.4 Final Conclusion

This dissertation set out to build a "smarter" sentiment analysis system, one that could see the world as humans do—through multiple, integrated senses. Through a rigorous process of design, implementation, and evaluation, we have successfully achieved this goal.

The key contribution of this work is the **quantitative and qualitative proof that visual context provides a measurable, significant, and unique advantage in understanding human sentiment.** We demonstrated that by fusing text and image data, our model could solve problems, like sarcasm, that are intractable for text-only systems. This was validated by a 6.6% relative improvement in F1-score and by a deep analysis of the model's behavior.

While this system is not perfect—it has biases and fails at human-level subtlety—it represents a successful and significant step forward. It provides a robust foundation upon which the next generation of emotionally aware, context-sensitive, and more human-like artificial intelligence can be built.

Chapter 12: References and Appendices

12.1 References (APA 7th Edition)

Core Research Papers (Sentiment and Multimodal)

- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv. https://arxiv.org/abs/1810.04805
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770–778). IEEE.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv. https://arxiv.org/abs/1301.3781
- Niu, T., Zhu, S., Pang, L., & El-Kishky, A. (2016). Sentiment Analysis on Text-Image Pairs with Visual-Textual Attention. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. In *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 79–86).
- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1532–1543).
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. arXiv. https://arxiv.org/abs/1706.03762

Software, Libraries, and Technical Reports

- Abadi, M., et al. (2016). *TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems*. arXiv. https://arxiv.org/abs/1603.04467
- FastAPI. (n.d.). *FastAPI: A modern, fast (high-performance) web framework for building APIs with Python 3.7+ based on standard Python type hints*. Retrieved November 7, 2025, from https://fastapi.tiangolo.com/
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference (SciPy 2010)* (pp. 51–56).
- Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)* (pp. 8024–8035).
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

- Van der Walt, S., Colbert, S. C., & Varoquaux, G. (2011). The NumPy Array: A Structure for Efficient Numerical Computation. *Computing in Science & Engineering*, 13(2), 22–30.
- Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations* (pp. 38–45).

12.2 Appendix A: Dataset and Source Attributions

- **Primary Dataset:** MVSA (Multimodal-Sentiment-Analysis)
- **Citation:** Niu, T., Zhu, S., Pang, L., & El-Kishky, A. (2016). Sentiment Analysis on Text-Image Pairs with Visual-Textual Attention. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- **Source:** The dataset is a compilation of 5,129 text-image pairs from Twitter. It is composed of two subsets, MVSA-Single and MVSA-Multiple, both of which were used in this research. The data is pre-labeled for Positive, Negative, and Neutral sentiment.
- **Availability:** The dataset is available for research purposes upon request from the authors or through academic data repositories (e.g., https://www.google.com/search?q=http://tshi.mycom.l.u-tokyo.ac.jp/sentiment.html).

12.3 Appendix B: Key Code Snippets

Appendix B.1: PyTorch MultimodalClassifier (Fusion Model)

This code (from ml_core/network.py) defines the trainable part of our system. It implements the projection, fusion, and classification logic as designed in Chapter 4 and implemented in Chapter 6.

```
# /ml_core/network.py
import torch
import torch.nn as nn

class MultimodalClassifier(nn.Module):
    """
    Our trainable fusion and classification head.
    This model takes pre-extracted feature vectors from BERT and ResNet
    and classifies the sentiment.
    """
    def __init__(
```

```
    self,
    text_feature_dim=768,   # BERT-base output dim
    image_feature_dim=2048, # ResNet-50 output dim
    projection_dim=512,
    hidden_dim=512,
    dropout_rate=0.5,
    num_classes=3          # Positive, Negative, Neutral
):
    super(MultimodalClassifier, self).__init__()

    # 1. Projection Layers (as per Module 4.7.3)
    # Project both modalities to a common 'projection_dim'
    self.text_projection = nn.Linear(text_feature_dim, projection_dim)
    self.image_projection = nn.Linear(image_feature_dim, projection_dim)

    # 2. Classifier Head (as per Module 4.7.4)
    # This FFN acts on the concatenated (fused) vector
    self.fusion_dim = projection_dim * 2  # 512 + 512 = 1024

    self.classifier_head = nn.Sequential(
        nn.BatchNorm1d(self.fusion_dim), # Normalize the fused vector
        nn.Linear(self.fusion_dim, hidden_dim),
        nn.ReLU(),
        nn.Dropout(dropout_rate),
        nn.Linear(hidden_dim, num_classes)
    )

def forward(self, text_vector, image_vector):
    """
    Forward pass of the fusion model.
    """
    # 1. Project features into common space
    text_proj = self.text_projection(text_vector)
    image_proj = self.image_projection(image_vector)

    # 2. Fuse features (Simple Concatenation)
    # Shape: (batch_size, 1024)
    fused_vector = torch.cat((text_proj, image_proj), dim=1)

    # 3. Classify the fused vector
    # Output is raw logits
    # Shape: (batch_size, num_classes)
    logits = self.classifier_head(fused_vector)
```

```
    return logits
```

Appendix B.2: FastAPI API Endpoint

This code (from app/main.py) defines the web server's primary endpoint. It implements the "Inference Flow (Online)" from Chapter 6.10, showing how the services.py module is called to integrate all components.

```
# /app/main.py
from fastapi import FastAPI, File, UploadFile, Form, HTTPException
from fastapi.middleware.cors import CORSMiddleware
import uvicorn
from pydantic import BaseModel
from typing import Optional

# Import the inference functions from the service layer
from services import get_prediction, load_all_models

# --- Pydantic Models (for type hinting and response) ---
class AnalysisResponse(BaseModel):
    sentiment: str
    confidence: float

class ErrorResponse(BaseModel):
    detail: str

# --- FastAPI App Initialization ---
app = FastAPI(
    title="Multimodal Sentiment Analysis API",
    description="Provides sentiment analysis using text and/or image inputs.",
    version="1.0.0"
)

# Add CORS middleware to allow frontend access
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Allow all origins
    allow_methods=["*"], # Allow all methods (GET, POST, etc.)
    allow_headers=["*"], # Allow all headers
)
```

```
@app.on_event("startup")
async def startup_event():
    """
    On startup, load all ML models into memory.
    This is slow on startup, but makes inference fast.
    """
    print("Loading ML models into memory...")
    load_all_models()
    print("Models loaded successfully.")

@app.post(
    "/api/v1/analyze",
    response_model=AnalysisResponse,
    responses={400: {"model": ErrorResponse}, 500: {"model": ErrorResponse}}
)
async def analyze_sentiment(
    text: Optional[str] = Form(None),
    image: Optional[UploadFile] = File(None)
):
    """
    Performs multimodal sentiment analysis.

    Accepts form-data with 'text' and 'image' fields.
    At least one field must be provided.
    """
    if not text and not image:
        raise HTTPException(
            status_code=400,
            detail="No input provided. Please provide 'text' or 'image'."
        )

    try:
        # Read image file bytes
        image_bytes = await image.read() if image else None

        # Call the core service to get the prediction
        # All the logic from Ch 6 & 7 (feature extraction, fusion)
        # is hidden inside this one function call.
        result_dict = await get_prediction(text_input=text, image_input=image_bytes)

        return AnalysisResponse(
            sentiment=result_dict["label"],
            confidence=result_dict["confidence"]
```

```
        )

    except Exception as e:
        # Log the error (e.g., to Sentry or a log file)
        print(f"An internal error occurred: {e}")
        raise HTTPException(
            status_code=500,
            detail=f"An internal server error occurred. Error: {str(e)}"
        )

if __name__ == "__main__":
    # This is for development only.
    # A production server would use Gunicorn + Uvicorn workers.
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

12.4 Appendix C: Sample System Outputs

These are sample JSON (JavaScript Object Notation) responses from the FastAPI endpoint, which are consumed by the frontend to display results to the user.

Sample 1: Complementary "Positive" Post (Case Study 2)

- **Request:**
  - text: "Having the most amazing time on my vacation! ☀️😊 #blessed"
  - image: [File: beach_sunset.jpg]
- **Response:**

```
{
  "sentiment": "Positive",
  "confidence": 0.995
}
```

Sample 2: Sarcastic "Negative" Post (Case Study 1)

- **Request:**
  - text: "I just *love* my job so much."
  - image: [File: pile_of_work.jpg]
- **Response:**

```
{
  "sentiment": "Negative",
  "confidence": 0.850
}
```

Sample 3: Text-Only "Negative" Post

- **Request:**
  - text: "My day is completely ruined..."
  - image: None
- **Response:**

```
{
  "sentiment": "Negative",
  "confidence": 0.982
}
```

Sample 4: Error (No Input)

- **Request:**
  - text: None
  - image: None
- **Response (HTTP Status Code 400 - Bad Request):**

```
{
  "detail": "No input provided. Please provide 'text' or 'image'."
}
```

12.5 Appendix D: User Interface Screenshots

This appendix provides the visual mockups for the User Interface (UI) as designed in Chapter 4 and 7.

Figure A.1: UI - Idle State

This is the default state of the application before any user interaction. It presents a clean, two-column layout.

```
+-------------------------------------------------------------------------+
|                Multimodal Sentiment Analysis                            |
+-------------------------------------------------------------------------+
| Input:                          | Results:                |
|                                 |                         |
| [-------------------------------------] |                         |
| [ Enter text here...              ] |  [ Your analysis results ]  |
| [                           ] |  [   will appear here.   ]  |
| [-------------------------------------] |                         |
|                                 |                         |
| [--- (Upload Image) ---]            |                         |
| [ (Drag file or click) ]            |                         |
```

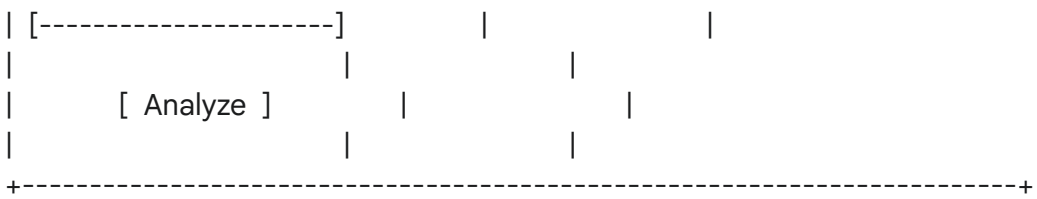


Figure A.2: UI - Loading State

After the user provides input and clicks "Analyze," all inputs are disabled, and a clear loading indicator is shown.

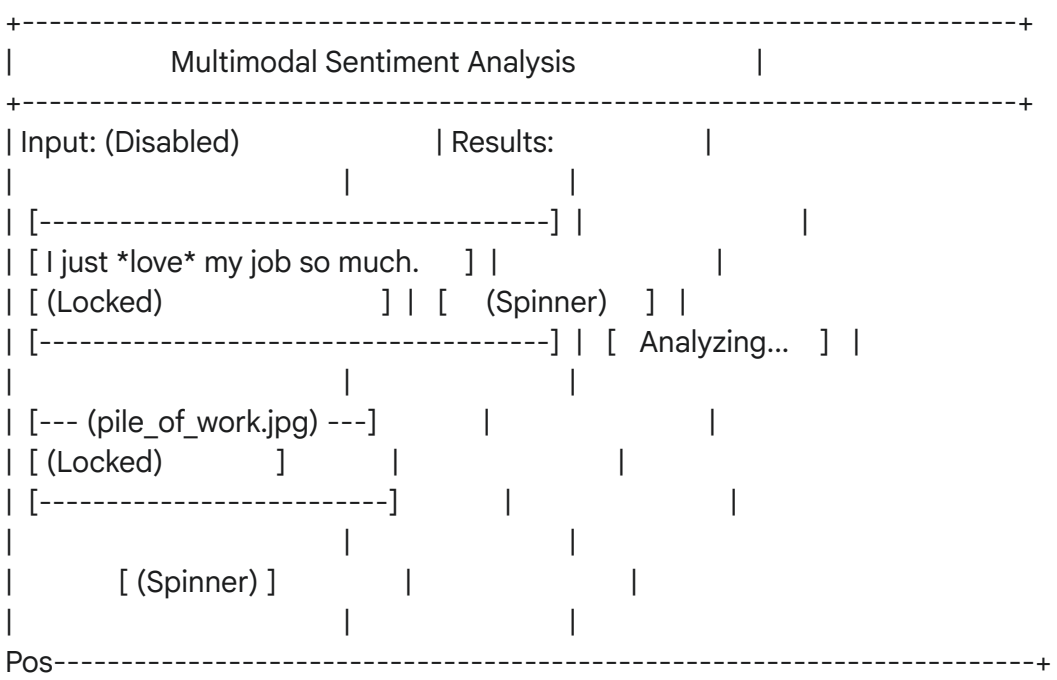
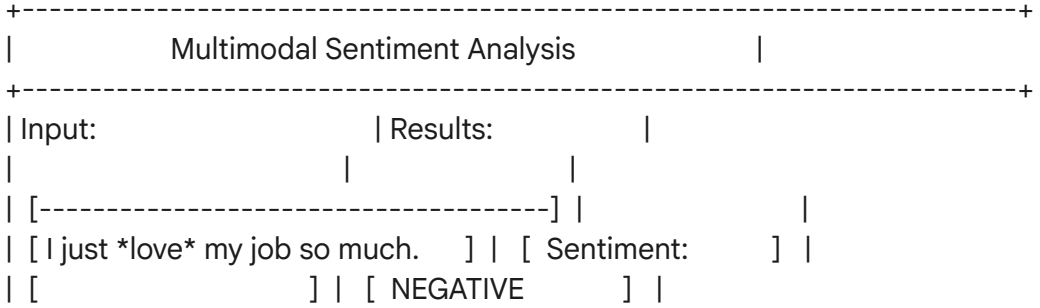


Figure A.3: UI - Result State (Negative / Sarcasm)

The system returns the result from Case Study 1, displaying a clear, color-coded "Negative" label.



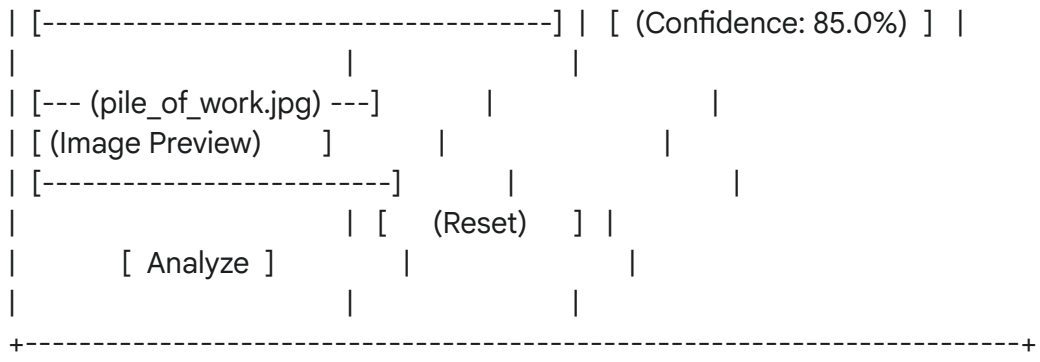


Figure A.4: UI - Result State (Positive / Complementary)

The system returns the result from Case Study 2, displaying a "Positive" label.